Clustering program

Generated by Doxygen 1.8.17

Chapter 1

Unit Testing Suite - File Index

1.1 Test Source Files

1.1.1 Core Test Programs (7 files)

1. **test_maths.f90**
   - Tests: 30 tests
   - Modules tested: `maths`
   - Features: Cross product, coordinate transformations, angle calculations, RMSD, center of geometry, distance calculations
   - Dependencies: maths.f90, mod_pdb.f90
2. **test_matrix.f90**
   - Tests: 10 tests
   - Modules tested: `mod_matrix`
   - Features: Matrix I/O, array I/O, Z-coordinate calculations, atoms distance calculations, angle calculations, RMSD matrix properties
   - Dependencies: mod_matrix.f90, maths.f90, mod_pdb.f90
3. **test_threshold.f90**
   - Tests: 9 tests (updated from 6)
   - Modules tested: `mod_threshold`
   - Features: Z-coordinate arrays, minimum distance, angle calculations, array sorting
   - Dependencies: mod_threshold.f90, maths.f90
4. **test_pdb.f90**
   - Tests: 11 tests
   - Modules tested: `mod_pdb`
   - Features: PDB structures, atoms, residues, chains, PDB I/O
   - Dependencies: mod_pdb.f90
5. **test_clustering.f90**
   - Tests: 8 tests
   - Modules tested: `mod_clust_algorithm`

- Features: Clustering statistics, simple clustering, mean linkage, min linkage, max linkage, cluster output writing
- Dependencies: mod_clust_algorithm.f90, read_input.f90, mod_pdb.f90, mod_assoc.f90, maths.f90

6. **test_assoc.f90**
   - Tests: 11 tests
   - Modules tested: `mod_assoc`
   - Features: Complexes I/O, object allocation, file parsing
   - Dependencies: mod_assoc.f90
7. **test_read_input.f90**
   - Tests: 10 tests
   - Modules tested: `read_input` (wrapper module)
   - Features: Input file reading, structure creation, error handling
   - Dependencies: read_input.f90, mod_pdb.f90, mod_assoc.f90

1.2 Documentation Files

1. **TESTING.md**
   - Comprehensive testing guide with all 7 test files
   - Compilation instructions for each test
   - Updated test coverage table (89 tests total)
   - Best practices and troubleshooting
   - Instructions for adding new tests
2. **TEST_SUMMARY.md**
   - Overview of entire test suite with 89 tests
   - Summary of all 7 test programs
   - Test coverage matrix with new modules
   - Example output
   - Overall statistics table
3. **QUICK_TEST.md**
   - Quick reference for running all 7 test suites
   - Command summary table with new tests
   - File overview with test counts
   - Pro tips
4. **README.md**
   - Test framework overview
   - Directory structure with all 7 test programs
   - Quick start guide
   - Test coverage table (89 tests)
   - Build requirements and verification
5. **INDEX.md** (this file)
   - File listing and descriptions for all 7 test programs
   - Documentation index with 5 files

1.3 Automation Files

1. **run_all_tests.sh**
    - Bash script to compile and run all 7 tests
    - Automatic dependency tracking for new tests
    - Summary reporting for all test programs
    - Exit code handling (0 = success, 1 = failure)

1.4 Modified Files

1. **Makefile**
    - Added test targets for test_assoc and test_read_input
    - Updated `test` target to run all 7 tests
    - Updated `clean_tests` target for new executables
    - Integrated with existing build system

1.5 Quick Start

```
cd clustering_program/src/
# Run all tests
make test
# Or use the automated script
bash run_all_tests.sh
# Individual test
make test_maths
make test_pdb
```

1.6 Test Distribution

```
Mathematical Operations (30 tests)
- cross product
- coordinate transformations
- angle calculations
- RMSD calculations
- center of geometry
- distance calculations
Matrix Operations (10 tests)
- matrix I/O
- array I/O
- z-coordinate matrix
- RMSD matrix
- atom distance matrix
- angle matrix
Array/Threshold Operations (9 tests)
- z-coordinate arrays
- minimum distance
- angle calculations
- array sorting
PDB Data Structures (11 tests)
- allocation
- atom properties
- residue management
- chain management
- PDB routines
Clustering Algorithms (8 tests)
- clustering statistics
- simple clustering
- linkage clustering
- cluster output
Association File Reading (11 tests)
- file reading
- object allocation
- file parsing
Read Input Module (10 tests)
- wrapper functions
- error handling
```

1.7 Testing Workflow

```
1. Build Tests (make test)
   |
   v
2. Compile each test program
   - Dependency resolution
   - Link with required modules
   |
   v
3. Execute tests sequentially
   - Each test reports pass/fail
   - Detailed error messages on failure
   - Summary statistics
   |
   v
4. Report results
   - Total tests run
   - Passed count
   - Failed count
   - Exit code (0=success, 1=failure)
```

1.8 Files Generated by Tests

When tests run, the following executables are created (removed by `make clean_tests`):

- `test_maths`
- `test_matrix`
- `test_threshold`
- `test_pdb`
- `test_clustering`

1.9 Support & References

- **TESTING.md** - Detailed guide with troubleshooting
- **TEST_SUMMARY.md** - Full overview and future enhancements
- **QUICK_TEST.md** - Common commands and tips

Created: February 2026 **Total Tests**: 89 **Coverage**: 7 modules, 26 functions

Chapter 2

Quick Reference: Running Unit Tests

2.1 TL;DR

```
cd clustering_program/src/
make test          # Compile and run all 89 tests
```

2.2 Test Commands

Command	Description
`make test`	Compile and run all 89 tests (7 test programs)
`make test_maths`	Test mathematical operations (30 tests)
`make test_matrix`	Test matrix distance generator (10 tests)
`make test_threshold`	Test threshold filtering (9 tests)
`make test_pdb`	Test PDB reading (11 tests)
`make test_clustering`	Test clustering algorithms (8 tests)
`make test_assoc`	Test association file reading (11 tests)
`make test_read_input`	Test read_input module (10 tests)
`make clean_tests`	Remove test executables
`bash run_all_tests.sh`	Run all tests with detailed output

2.3 Test Files

File	Tests	Lines	Purpose
test_maths.f90	30	850	Cross product, transformations, RMSD, angles, COG, distance
test_matrix.f90	10	370	Matrix I/O, array I/O, Z-coord, atom dist, angle, RMSD
test_threshold.f90	9	375	Z-coord, min distance, angles, sorting
test_pdb.f90	11	210	PDB I/O, PDB structures, chains, residues, atoms
test_clustering.f90	8	275	Statistics, clustering, output writing
test_assoc.f90	11	380	Complexes I/O, allocation, parsing
test_read_input.f90	10	355	Input wrapper, error handling

2.4 Expected Output

All tests pass with output like:

```
[PASS] Test 1: Test description
[PASS] Test 2: Another test
========================================
TEST SUMMARY
========================================
Total tests:  N
```

```
Passed:       N
Failed:       0
========================================
```

2.5 Individual Compilation

```
# Manual compilation (if not using Make)
gfortran -o test_maths test_maths.f90 maths.f90 mod_pdb.f90
./test_maths
gfortran -fopenmp -o test_matrix test_matrix.f90 mod_matrix.f90 maths.f90 mod_pdb.f90
./test_matrix
gfortran -fopenmp -o test_threshold test_threshold.f90
./test_threshold
gfortran -o test_pdb test_pdb.f90 mod_pdb.f90
./test_pdb
gfortran -fopenmp -o test_clustering test_clustering.f90
./test_clustering
```

2.6 Documentation

- TEST_SUMMARY.md - Overview of test suite
- TESTING.md - Detailed testing guide

2.7 Key Features

[x] 89 comprehensive tests [x] 7 test programs (added test_assoc.f90 and test_read_input.f90) [x] All major modules covered [x] OpenMP compatible [x] No external data files needed [x] Fast execution ($<$ 2 seconds) [x] Clear pass/fail reporting [x] Easy to extend

Pro Tip: Use `make test 2>&1 | tee test_results.log` to save test output to a file.

Chapter 3

Unit Testing Framework

This directory contains the comprehensive unit testing framework for the clustering program.

3.1 Contents

3.1.1 Test Programs

- **test_maths.f90** - Tests for mathematical operations (30 tests)
- **test_matrix.f90** - Tests for matrix operations (10 tests)
- **test_threshold.f90** - Tests for threshold calculations (9 tests)
- **test_pdb.f90** - Tests for PDB file handling (11 tests)
- **test_clustering.f90** - Tests for clustering algorithms (8 tests)
- **test_assoc.f90** - Tests for association file reading (11 tests)
- **test_read_input.f90** - Tests for read_input module (10 tests)

3.1.2 Test Automation

- **run_all_tests.sh** - Automated script to run all unit tests

3.1.3 Documentation

Document	Purpose	Best For
QUICK_TEST.md	Quick reference guide	Getting started quickly
TESTING.md	Comprehensive testing guide	Understanding all details
TEST_SUMMARY.md	Test suite overview	High-level overview
INDEX.md	File index and statistics	Navigation

3.2 Quick Start

3.2.1 Run all tests

```
cd ../src
make test
```

3.2.2 Run specific test

```
cd ../src
make test_maths    # or test_matrix, test_threshold, test_pdb, test_clustering
```

3.2.3 Clean test executables

```
cd ../src
make clean_tests
```

3.3 Where to Start?

1. **New to testing?** - Start with QUICK_TEST.md
2. **Need details?** - Read TESTING.md
3. **Want overview?** - See TEST_SUMMARY.md
4. **Need file list?** - Check INDEX.md

3.4 Test Coverage

Module	Tests	Status
maths.f90	30	Success
mod_matrix.f90	10	Success
mod_threshold.f90	9	Success
mod_pdb.f90	11	Success
mod_clust_algorithm.f90	8	Success
mod_assoc.f90	11	Success
read_input.f90	10	Success
Total	**89**	**Success**

3.5 Build Requirements

- gfortran compiler
- LAPACK and BLAS libraries
- OpenMP support

3.6 Verification

All 89 unit tests are defined and ready to run. The framework provides:

- Comprehensive test coverage across 7 test files
- Assertion helpers for test validation
- Pass/fail reporting with detailed output
- Error message display
- Command-line integration via Makefile
- Standalone bash script runner

3.7 Notable Behavior

Array Parameter in Clustering:

- The `-array` option is **mandatory** for all clustering methods **except RMSD**
- When using `-matrix_type rmsd`, the array parameter is **optional**

- When an array is provided, complexes are sorted by the array values
- If array is not provided for non-RMSD methods, an error is raised
- Without an array, cluster averages and standard deviations are computed from the matrix

Test File Coverage Updates (February 2026):

- Added comprehensive tests for mod_assoc module (association file reading)
- Added comprehensive tests for read_input module (input file processing)
- Expanded maths module tests from 2 to 30 tests with detailed coordinate transformation coverage
- Updated all test documentation to reflect current test count (89 total tests)

For detailed information, see TESTING.md.

Chapter 4

Unit Testing Suite for Clustering Program

4.1 Summary

A comprehensive unit testing framework has been created for the Fortran clustering program with **7 test programs** covering **89 individual tests** across all major modules.

4.2 Test Programs Created

4.2.1 1. test_maths.f90 (30 tests)

- Cross product calculation (basis vectors, anti-commutativity, self-product, arbitrary vectors) - 4 tests
- Coordinate transformations (identity, translation, rotation, combinations) - 7 tests
- Angle calculations in 3D and 2D - 8 tests
- RMSD calculations - 3 tests
- Center of geometry computations - 4 tests
- Distance calculations - 4 tests
- Tests fundamental mathematical operations used throughout the program

4.2.2 2. test_matrix.f90 (10 tests)

- Matrix file I/O operations (write/read matrix) - 2 tests
- Array file I/O operations (write/read array) - 2 tests
- Z-coordinate matrix calculations - 2 tests
- RMSD matrix calculations with symmetry and diagonal verification - 2 tests
- Atom distance matrix calculations - 1 test
- Angle matrix calculations - 1 test
- Tests core matrix operations for encounter data

4.2.3 3. test_threshold.f90 (9 tests)

- Z-coordinate array calculations - 2 tests
- Minimum distance computations - 2 tests
- Angle calculations across arrays - 2 tests
- Array sorting with index tracking - 3 tests
- Tests threshold computation infrastructure

4.2.4 4. test_pdb.f90 (11 tests)

- PDB structure initialization and allocation - 3 tests
- Atom properties (coordinates, names, residue info) - 2 tests
- Residue management - 2 tests
- Chain handling - 2 tests
- PDB count/allocate/fill routines - 2 tests
- Tests PDB data structure integrity

4.2.5 5. test_clustering.f90 (8 tests)

- Clustering statistics (mean distance, standard deviation) - 2 tests
- Simple clustering algorithm with tight cluster detection - 2 tests
- Minimum linkage clustering - 1 test
- Maximum linkage clustering - 1 test
- Mean linkage clustering - 1 test
- Cluster output file writing and validation - 1 test
- Tests core clustering algorithm components

4.2.6 6. test_assoc.f90 (11 tests)

- Association/complexes file reading - 6 tests
- Association object allocation - 3 tests
- File size and line counting - 2 tests
- Tests association file I/O and data structures

4.2.7 7. test_read_input.f90 (10 tests)

- Read input wrapper functions - 7 tests
- Error handling and edge cases - 3 tests
- Tests input file processing module

4.3 Files

```
clustering_program/unit_tests/
- test_maths.f90           # Mathematical operations tests (30 tests)
- test_matrix.f90          # Matrix operations tests (10 tests)
- test_threshold.f90       # Threshold array tests (9 tests)
- test_pdb.f90             # PDB structure tests (11 tests)
- test_clustering.f90      # Clustering algorithm tests (8 tests)
- test_assoc.f90           # Association file tests (11 tests) [NEW]
- test_read_input.f90      # Read input module tests (10 tests) [NEW]
- run_all_tests.sh         # Automated test runner script
- INDEX.md                 # Documentation index
- QUICK_TEST.md            # Quick test reference
- TESTING.md               # Comprehensive testing documentation
- README.md                # Test suite overview
- TEST_SUMMARY.md          # This file
```

4.4 Building Tests

4.4.1 Individual tests:

```
cd clustering_program/src/
# Test specific module
gfortran -fopenmp -o test_maths ../unit_tests/test_maths.f90 maths.o mod_pdb.o
gfortran -fopenmp -o test_matrix ../unit_tests/test_matrix.f90 mod_matrix.o maths.o mod_pdb.o
gfortran -fopenmp -o test_threshold ../unit_tests/test_threshold.f90 mod_threshold.o maths.o
gfortran -o test_pdb ../unit_tests/test_pdb.f90 mod_pdb.o
gfortran -fopenmp -o test_clustering ../unit_tests/test_clustering.f90 mod_clust_algorithm.o read_input.o
       mod_pdb.o mod_assoc.o maths.o
gfortran -o test_assoc ../unit_tests/test_assoc.f90 mod_assoc.o
gfortran -o test_read_input ../unit_tests/test_read_input.f90 read_input.o mod_pdb.o mod_assoc.o
```

4.4.2 Run all tests via Make:

```
cd clustering_program/src/
make test                # Compile and run all 7 test suites
make test_maths          # Run specific test
make test_matrix
make test_threshold
make test_pdb
make test_clustering
make test_assoc          # NEW
make test_read_input     # NEW
make clean_tests         # Remove test executables only
```

4.4.3 Automated test runner:

```
cd clustering_program/unit_tests/
bash run_all_tests.sh
```

4.5 Test Coverage Matrix

Module	Function	Tests	Type
maths	cross	4	Unit
maths	update_complex	2	Unit
mod_matrix	allocation	2	Unit
mod_matrix	symmetry	2	Unit
mod_threshold	statistics	3	Unit
mod_threshold	bounds	3	Unit
mod_pdb	allocation	3	Unit
mod_pdb	atoms	3	Unit
mod_pdb	residues	2	Unit
mod_clustering	distances	3	Unit
mod_clustering	helpers	3	Unit

Total: 89 unit tests

4.6 Test Features

[x] **Comprehensive coverage** - Tests mathematical operations, data structures, and algorithms [x] **Floating-point tolerance** - Uses appropriate precision (1d-10 to 1d-8) for double precision [x] **Edge case handling** - Tests boundary conditions and special cases [x] **Detailed reporting** - Pass/fail output with detailed diagnostics [x] **Automated execution** - Makefile targets and shell script runner [x] **OpenMP compatible** - Handles parallel compilation flags [x] **Modular design** - Independent test programs, easy to extend

4.7 Example Test Output

```
========================================
```

```
UNIT TESTS FOR MATHS MODULE
========================================
Testing cross product...
  [PASS] Test 1: i x j = k
  [PASS] Test 2: j x i = -k
  [PASS] Test 3: v x v = 0
  [PASS] Test 4: Arbitrary vectors
Testing coordinate transformation (update_complex)...
  [PASS] Test 1: Identity transformation
  [PASS] Test 2: Pure translation
========================================
TEST SUMMARY
========================================
Total tests:  30
Passed:       30
Failed:       0
========================================
```

4.8 Example Output

```
========================================
UNIT TESTS FOR MATHS MODULE
========================================
Testing cross product...
  [PASS] Test 1: i x j = k
  [PASS] Test 2: j x i = -k
  [PASS] Test 3: v x v = 0
  [PASS] Test 4: Arbitrary vectors
Testing coordinate transformation (update_complex)...
  [PASS] Test 1: Identity transformation
  [PASS] Test 2: Pure translation
  [PASS] Test 3: XC1-only transformation
  [PASS] Test 4: XC2-only transformation
  [PASS] Test 5: Rotation-only transformation
  [PASS] Test 6: Combined transformations
  [PASS] Test 7: Matrix operation verification
... (30 total tests in test_maths.f90)
========================================
TEST SUMMARY FOR MATHS
========================================
Total tests:  30
Passed:       30
Failed:       0
========================================
```

4.9 Overall Statistics

Test File	Tests	Lines	Status
test_maths.f90	30	850	Success
test_matrix.f90	10	370	Success
test_threshold.f90	9	375	Success
test_pdb.f90	11	210	Success
test_clustering.f90	8	Success	Success
test_assoc.f90	11	380	Success
test_read_input.f90	10	355	Success
TOTAL	**89**	**2,815**	**YES**

4.10 Running Tests

4.10.1 Quick test:

```
cd clustering_program/src/
make test          # Runs all 89 tests across 7 test programs
```

4.10.2 Run specific test:

```
cd clustering_program/src/
make test_maths    # Or test_matrix, test_threshold, test_pdb, etc.
make test_assoc    # Run new association tests
make test_read_input  # Run new read_input tests
```

4.10.3 Verbose output:

```
cd clustering_program/unit_tests/
bash run_all_tests.sh
```

4.10.4 Individual test debugging:

```
cd clustering_program/src/
gfortran -fopenmp -o test_matrix ../unit_tests/test_matrix.f90 mod_matrix.o maths.o mod_pdb.o
./test_matrix
```

4.11 Documentation

See TESTING.md for:

- Detailed test descriptions
- Compilation instructions
- Troubleshooting guide
- Instructions for adding new tests
- Best practices and references

4.12 Future Enhancements

- [ ] Integration tests for multi-module workflows
- [ ] Performance benchmarks
- [ ] Code coverage analysis
- [ ] Continuous integration (GitHub Actions)
- [ ] Memory leak detection (valgrind)
- [ ] Additional clustering algorithm tests

4.13 Dependencies

- gfortran (or other Fortran compiler)
- OpenMP (for parallel tests)
- LAPACK/BLAS (existing project dependency)

4.14 Notes

- Tests use standard IEEE 754 double precision (kind=8)
- All tests are deterministic and reproducible
- Tests do not require external data files
- Clean exit codes: 0 (success), 1 (failure)

Created: February 2026
Author: Abraham Muñiz-Chicharro
Version: 1.0

Chapter 5

Unit Testing Guide for Clustering Program

5.1 Overview

This directory contains comprehensive unit tests for the Fortran subroutines in the clustering program. The tests verify correctness of mathematical operations, data structures, and algorithmic components.

5.2 Test Files

5.2.1 1. test_maths.f90

Tests mathematical operations in the `maths` module:

- **Cross product calculation** (`cross` subroutine) - 4 tests
  - Standard basis vector products (i x j = k)
  - Anti-commutativity (v x w = -w x v)
  - Self-product is zero (v x v = 0)
  - Arbitrary vector products
- **Coordinate transformations** (`update_complex` subroutine) - 7 tests
  - Identity transformations
  - Pure translation
  - XC1-only transformation
  - XC2-only transformation
  - Rotation-only transformation
  - Combined transformations
  - Verification of matrix operations
- **Angle calculations** (`vectors_angle_3D`, `vectors_angle_2D`) - 8 tests
  - 3D angle calculations with various vector combinations
  - 2D angle calculations with perpendicular and parallel vectors
- **RMSD calculations** (`rmsd` subroutine) - 3 tests
  - Identical structures (zero RMSD)
  - Translated structures
  - Rotated structures
- **Center of geometry** (`calculate_cog` subroutine) - 4 tests
  - Simple geometries

  - Symmetric point clouds
  - Offset coordinates

- **Distance calculations** (`calculate_distance` subroutine) - 4 tests
  - Unit vectors
  - Arbitrary vectors
  - Zero distance cases

Total: 30 tests

5.2.2 2. test_matrix.f90

Tests matrix operations in the `mod_matrix` module:

- **Matrix file I/O** (`write_matrix`, `read_matrix`) - 2 tests
  - Small matrices (5 x 5)
  - Symmetric matrix properties
- **Array file I/O** (`write_array`, `read_array`) - 2 tests
  - Single and double precision arrays
  - Large array serialization
- **Z-coordinate matrix calculation** (`matrix_z_coord`) - 2 tests
  - Identity transformation (coordinates unchanged)
  - Translation transformations
- **RMSD matrix calculation** (`matrix_rmsd`) - 2 tests
  - Symmetry verification
  - Diagonal properties (zeros on diagonal)
- **Atom distance matrix calculation** (`matrix_atoms_dist`) - 1 test
  - Minimum distance differences
- **Angle matrix calculation** (`matrix_angle`) - 1 test
  - 2D angle differences

Total: 10 tests

5.2.3 3. test_threshold.f90

Tests threshold array operations in the `mod_threshold` module:

- **Z-coordinate array calculation** (`array_z_coord`) - 2 tests
  - Identity transformation
  - Translation effects
- **Minimum distance calculation** (`array_atoms_dist`) - 2 tests
  - Distance computations
  - Minimum value detection
- **Angle calculations** (`array_angle`) - 2 tests
  - 3D angle calculations across arrays
  - Multiple vector pair angles

- **Array sorting** (`sort_array`) - 3 tests
  - Ascending order sorting
  - Index tracking
  - Large array handling

Total: 9 tests

5.2.4 4. test_pdb.f90

Tests PDB file structures and operations in the `mod_pdb` module:

- **PDB type initialization** - 3 tests
  - File structure allocation
  - Atom array creation
  - Residue array creation
- **Atom properties** - 2 tests
  - Coordinate assignment
  - Name and residue information
- **Residue management** - 2 tests
  - Residue property assignment
  - Atom array allocation within residues
- **Chain management** - 2 tests
  - Chain property assignment
  - Atom/residue array allocation
- **PDB routines** - 2 tests
  - count_atoms/count_residues/count_chains
  - allocate_pdb_object/fill_pdb_object

Total: 11 tests

5.2.5 5. test_clustering.f90

Tests clustering algorithms in the `mod_clust_algorithm` module:

- **Clustering statistics** (`calculate_mean_distance`, `calculate_std_deviation`) - 2 tests
  - Mean distance calculations from clusters
  - Standard deviation computations
- **Simple clustering algorithm** (`simple_clustering`) - 2 tests
  - Tight cluster pair detection
  - Identical point handling
- **Linkage clustering** (`linkage_clustering`) - 3 tests
  - Minimum linkage
  - Maximum linkage
  - Mean linkage
- **Cluster output** (`write_clusters`) - 1 test
  - File output verification
  - Cluster format validation

Total: 8 tests

5.2.6 6. test_assoc.f90 (NEW)

Tests association/complexes file reading in the mod_assoc module:

- **File reading** (read_assoc_file) - 6 tests
  - Successfully read complexes from files
  - Correct structure allocation
  - Data verification (natom, residues, chains)
  - Header line parsing
  - Comment line handling
  - Sequential file reading
- **Object allocation** (allocate_assoc_object) - 3 tests
  - Memory allocation
  - Array initialization
  - Size verification
- **File parsing** (size_assoc) - 2 tests
  - Line counting with comment handling
  - Empty file handling

Total: 11 tests

5.2.7 7. test_read_input.f90 (NEW)

Tests read_input module wrapper functions:

- **Association file reading** (read_input_assoc) - 7 tests
  - File reading and structure creation
  - Data verification (natom, residues, chains)
  - Header and comment handling
  - Sequential file operations
  - Memory allocation
  - File existence checking
  - Data integrity validation
- **Error handling** - 3 tests
  - Non-existent file handling
  - Invalid file format detection
  - Sequential read verification

Total: 10 tests

5.3 Compilation

5.3.1 Compile individual tests:

```
# Test mathematical operations (requires maths.f90 and mod_pdb.f90)
gfortran -fopenmp -o test_maths test_maths.f90 ../src/maths.o ../src/mod_pdb.o
# Test matrix operations (requires mod_matrix.f90, maths.f90, mod_pdb.f90)
gfortran -fopenmp -o test_matrix test_matrix.f90 ../src/mod_matrix.o ../src/maths.o ../src/mod_pdb.o
# Test threshold operations (requires mod_threshold.f90, maths.f90)
gfortran -fopenmp -o test_threshold test_threshold.f90 ../src/mod_threshold.o ../src/maths.o
# Test PDB operations (requires mod_pdb.f90)
gfortran -o test_pdb test_pdb.f90 ../src/mod_pdb.o
# Test clustering operations (requires mod_clust_algorithm.f90, read_input.f90, etc.)
gfortran -fopenmp -o test_clustering test_clustering.f90 ../src/mod_clust_algorithm.o ../src/read_input.o
       ../src/mod_pdb.o ../src/mod_assoc.o ../src/maths.o
# Test association file reading (requires mod_assoc.f90)
gfortran -o test_assoc test_assoc.f90 ../src/mod_assoc.o
# Test read_input module (requires read_input.f90, mod_pdb.f90, mod_assoc.f90)
gfortran -o test_read_input test_read_input.f90 ../src/read_input.o ../src/mod_pdb.o ../src/mod_assoc.o
```

5.3.2 Compile all tests at once:

```
# Using make (from src directory)
make test
# Or using the test runner script (from unit_tests directory)
bash run_all_tests.sh
```

5.4 Running Tests

5.4.1 Run individual tests:

```
./test_maths
./test_matrix
./test_threshold
./test_pdb
```

5.4.2 Run all tests:

```
# Create a simple test runner script
bash run_all_tests.sh
```

5.5 Example Output

```
========================================
UNIT TESTS FOR MATHS MODULE
========================================
Testing cross product...
  [PASS] Test 1: i x j = k
  [PASS] Test 2: j x i = -k
  [PASS] Test 3: v x v = 0
  [PASS] Test 4: Arbitrary vectors
Testing coordinate transformation (update_complex)...
  [PASS] Test 1: Identity transformation
  [PASS] Test 2: Pure translation
========================================
TEST SUMMARY
========================================
Total tests:  30
Passed:       30
Failed:       0
========================================
```

5.6 Test Coverage

Module	Subroutine	Tests	Status
maths	cross	4	Success
maths	update_complex	7	Success
maths	vectors_angle_3D	4	Success
maths	vectors_angle_2D	4	Success
maths	rmsd	3	Success
maths	calculate_cog	4	Success
maths	calculate_distance	4	Success
mod_matrix	write_matrix / read_matrix	2	Success
mod_matrix	write_array / read_array	2	Success
mod_matrix	matrix_z_coord	2	Success
mod_matrix	matrix_rmsd_calc	2	Success
mod_matrix	matrix_atoms_dist	1	Success
mod_matrix	matrix_angle	1	Success
mod_threshold	array_z_coord	2	Success
mod_threshold	array_atoms_dist	2	Success
mod_threshold	array_angle	2	Success
mod_threshold	sort_array	3	Success

Module	Subroutine	Tests	Status
mod_pdb	PDB allocation	3	Success
mod_pdb	atom properties	2	Success
mod_pdb	residue management	2	Success
mod_pdb	chain management	2	Success
mod_pdb	count/allocate/fill	2	Success
mod_clust_algorithm	clustering statistics	2	Success
mod_clust_algorithm	simple_clustering	2	Success
mod_clust_algorithm	linkage_clustering	3	Success
mod_clust_algorithm	write_clusters	1	Success
mod_assoc	read_assoc_file	6	Success
mod_assoc	allocate_assoc_object	3	Success
mod_assoc	size_assoc	2	Success
read_input	read_input_assoc	7	Success
read_input	error_handling	3	Success

Total: 89 tests

5.7 Adding New Tests

To add tests for additional subroutines:

1. **Create a new test program** following the template:
```
program test_my_module
    implicit none
    integer :: num_tests, num_passed, num_failed

    num_tests = 0
    num_passed = 0
    num_failed = 0

    ! Call test subroutines
    call test_my_subroutine(num_tests, num_passed, num_failed)

    ! Print summary
    if (num_failed > 0) stop 1
end program
```
2. **Implement test subroutines** with clear assertions:
```
subroutine test_my_subroutine(num_tests, num_passed, num_failed)
    ! Test implementation
end subroutine
```
3. **Add compilation rule** to Makefile
4. **Document test** in this README

5.8 Troubleshooting

5.8.1 Runtime failures:

- Check for NaN/Inf values in calculations
- Verify array bounds and allocation
- Review floating-point tolerance settings

5.8.2 Performance issues:

- Monitor memory usage for large matrices
- Use OpenMP pragmas for parallelized loops

5.9 References

- Fortran Standard Library
- OpenMP Fortran API
- PDB File Format

Author: Abraham Muñiz-Chicharro
Version: 1.0
Date: February 2026

Chapter 6

Modules Index

6.1 Modules List

Here is a list of all modules with brief descriptions:

Chapter 7

Data Type Index

7.1 Data Types List

Here are the data types with brief descriptions:

Chapter 8

File Index

8.1 File List

Here is a list of all files with brief descriptions:

Chapter 9

Module Documentation

9.1 maths Module Reference

Mathematical utilities and coordinate transformation routines.

Functions/Subroutines

- subroutine cross (v1, v2, v3)

  Compute the cross product of two 3D vectors.
- subroutine update_complex (xc1, xc2, trans, rot_vx, rot_vy, nb_atoms, coords, new_coords)

  Apply translation and rotation to a set of atomic coordinates.
- subroutine vectors_angle_3d (vector1, vector2, theta_degrees)

  Compute the angle in degrees between two 3D vectors.
- subroutine vectors_angle_2d (vector1, vector2, theta_degrees)

  Compute the angle in degrees between two vectors projected to 2D.
- subroutine rmsd (rmsd_value, nb_atoms, coords1, coords2)

  Compute the root-mean-square deviation (RMSD) between two sets of coordinates.
- subroutine calculate_cog (cog, coords, nb_atoms)

  Compute the center of geometry (centroid) of a set of points.
- subroutine calculate_distance (p1, p2, dist)

  Compute Euclidean distance between two 3D points.

9.1.1 Detailed Description

Mathematical utilities and coordinate transformation routines.
This module provides fundamental math operations including cross products, coordinate transformations, angle calculations, and rotation matrices.

Author

Abraham Muñiz-Chicharro

9.1.2 Function/Subroutine Documentation

9.1.2.1 calculate_cog()

```
subroutine maths::calculate_cog (
             real (kind=8), dimension(3), intent(out) cog,
             real (kind=8), dimension(nb_atoms, 3), intent(in) coords,
             integer, intent(in) nb_atoms )
```

Compute the center of geometry (centroid) of a set of points.
Sums the coordinates and divides by the number of points to produce the centroid.

Parameters

out	*cog*	Centroid vector (3 elements)
in	*coords*	Input coordinates, shape (nb_atoms,3)
in	*nb_atoms*	Number of atoms / points

9.1.2.2 calculate_distance()

```
subroutine maths::calculate_distance (
            real (kind=8), dimension(3), intent(in) p1,
            real (kind=8), dimension(3), intent(in) p2,
            real (kind=8), intent(out) dist )
```

Compute Euclidean distance between two 3D points.

Parameters

in	*p1*	First point (3 elements)
in	*p2*	Second point (3 elements)
out	*dist*	Euclidean distance ||p1 - p2||

9.1.2.3 cross()

```
subroutine maths::cross (
            real ( kind=8 ), dimension ( 3 ), intent(in) v1,
            real ( kind=8 ), dimension ( 3 ), intent(in) v2,
            real ( kind=8 ), dimension ( 3 ), intent(out) v3 )
```

Compute the cross product of two 3D vectors.
Computes v3 = v1 x v2 using the standard right-hand rule.

Parameters

in	*v1*	Left-hand operand vector (3 elements)
in	*v2*	Right-hand operand vector (3 elements)
out	*v3*	Resulting cross product (3 elements)

9.1.2.4 rmsd()

```
subroutine maths::rmsd (
            real(kind=8), intent(out) rmsd_value,
            integer, intent(in) nb_atoms,
            real(kind=8), dimension(:, :), intent(in) coords1,
            real(kind=8), dimension(:, :), intent(in) coords2 )
```

Compute the root-mean-square deviation (RMSD) between two sets of coordinates.
Calculates RMSD = sqrt((1/N) $*$ sum_i ||coords1(i,:) - coords2(i,:)||^2).

Parameters

out	*rmsd_value*	Resulting RMSD (scalar)
in	*nb_atoms*	Number of atoms / points (integer)
in	*coords1*	First coordinate set, shape (nb_atoms,3)

Parameters

in	*coords2*	Second coordinate set, shape (nb_atoms,3)

9.1.2.5 update_complex()

```
subroutine maths::update_complex (
            real ( kind = 8 ), dimension ( 3 ), intent(in) xc1,
            real ( kind = 8 ), dimension ( 3 ), intent(in) xc2,
            real ( kind = 8 ), dimension ( 3 ), intent(in) trans,
            real ( kind = 8 ), dimension ( 3 ), intent(in) rot_vx,
            real ( kind = 8 ), dimension ( 3 ), intent(in) rot_vy,
            integer nb_atoms,
            real ( kind = 8 ), dimension (:, :), intent(in) coords,
            real ( kind = 8 ), dimension (:, :), intent(out) new_coords )
```

Apply translation and rotation to a set of atomic coordinates.
Transforms the coordinates of a complex from a frame centered at `xc2` to a new frame centered at `xc1`. The routine first recenters coordinates by `xc2`, applies a rotation defined by `rot_vx` and `rot_vy` (their cross product defines the third axis), then applies an additional translation `trans`, and writes the results to `new_coords`.

Parameters

in	*xc1*	Target center position (3)
in	*xc2*	Original center position (3)
in	*trans*	Translation to apply after rotation (3)
in	*rot_vx*	First rotation axis vector (3)
in	*rot_vy*	Second rotation axis vector (3)
in	*nb_atoms*	Number of atoms (integer)
in	*coords*	Input coordinates, shape (nb_atoms,3)
out	*new_coords*	Output coordinates, shape (nb_atoms,3)

9.1.2.6 vectors_angle_2d()

```
subroutine maths::vectors_angle_2d (
            real( kind=8 ), dimension( 3 ), intent(in) vector1,
            real( kind=8 ), dimension( 3 ), intent(in) vector2,
            real (kind=8), intent(out) theta_degrees )
```

Compute the angle in degrees between two vectors projected to 2D.
Uses only the first two components of the input vectors to compute the angle in the XY plane. Returns 0 if either vector has zero length in the XY plane.

Parameters

in	*vector1*	First vector (use components 1 and 2)
in	*vector2*	Second vector (use components 1 and 2)
out	*theta_degrees*	Angle between projected vectors in degrees

9.1.2.7 vectors_angle_3d()

```
subroutine maths::vectors_angle_3d (
```

```
            real( kind=8 ), dimension( 3 ), intent(in) vector1,
            real( kind=8 ), dimension( 3 ), intent(in) vector2,
            real (kind=8), intent(out) theta_degrees )
```

Compute the angle in degrees between two 3D vectors.
Calculates the angle (in degrees) between `vector1` and `vector2` using the dot product formula and `acos`. No modification of inputs.

Parameters

in	*vector1*	First 3D vector (3 elements)
in	*vector2*	Second 3D vector (3 elements)
out	*theta_degrees*	Angle between vectors in degrees

9.2 mod_array Module Reference

Functions/Subroutines

- subroutine array_z_coord (complexes, array_sorted, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, solute↩ _crds)

  Build a matrix of absolute differences of Z-coordinates per encounter.
- subroutine array_atoms_dist (complexes, array_sorted, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, solute1_crds, solute2_crds)

  Build matrix from minimum inter-atomic distances between two solutes.
- subroutine array_angle (complexes, array_sorted, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, point1a, point1b, point2a, point2b, dimensions)

  Build matrix of angular differences between encounters.
- subroutine write_array (array, filename)
- subroutine read_array (array, n, filename)
- subroutine merge_sorted_segments (arr, complexes, tmp_arr, tmp_comp, left, mid, right)

  Merge two adjacent sorted segments of arrays in place.

9.2.1 Function/Subroutine Documentation

9.2.1.1 array_angle()

```
subroutine mod_array::array_angle (
            type(type_assoc_file), intent(inout) complexes,
            real(kind=8), dimension(n), intent(out) array_sorted,
            integer, intent(in) n,
            integer, intent(in) nb_atoms,
            real(kind=8), dimension(3), intent(in) xc1,
            real(kind=8), dimension(3), intent(in) xc2,
            real(kind=8), dimension(:, :), intent(in) trans_vector,
            real(kind=8), dimension(:, :), intent(in) rot1,
            real(kind=8), dimension(:, :), intent(in) rot2,
            real(kind=8), dimension(:), intent(in) point1a,
            real(kind=8), dimension(:), intent(in) point1b,
            real(kind=8), dimension(:), intent(in) point2a,
            real(kind=8), dimension(:), intent(in) point2b,
            integer, intent(in) dimensions )
```

Build matrix of angular differences between encounters.
Computes the angle (2D or 3D) for each encounter and fills the pairwise absolute difference matrix.

Parameters

out	*matrix*	Output (n,n) matrix of pairwise angular differences
out	*array*	Output (n) array with per-encounter angles
in	*n*	Number of encounters
in	*nb_atoms*	Number of atoms in solute 2 to transform
in	*xc1,xc2*	Solute centers used for translation transformations
in	*trans_vector*	Transform translation vectors (n,3)
in	*rot1,rot2*	Transform rotation arrays (n,3)
in	*point1a*	First point defining reference vector
in	*point1b*	Second point defining reference vector
in	*point2a*	First point of vector to be transformed
in	*point2b*	Second point of vector to be transformed
in	*dimensions*	Dimensionality of angle calculation (2 or 3)

9.2.1.2 array_atoms_dist()

```
subroutine mod_array::array_atoms_dist (
            type(type_assoc_file), intent(inout) complexes,
            real(kind=8), dimension(n), intent(out) array_sorted,
            integer, intent(in) n,
            integer, intent(in) nb_atoms,
            real(kind=8), dimension(3), intent(in) xc1,
            real(kind=8), dimension(3), intent(in) xc2,
            real(kind=8), dimension(:, :), intent(in) trans_vector,
            real(kind=8), dimension(:, :), intent(in) rot1,
            real(kind=8), dimension(:, :), intent(in) rot2,
            real(kind=8), dimension(:, :), intent(in) solute1_crds,
            real(kind=8), dimension(:, :), intent(in) solute2_crds )
```

Build matrix from minimum inter-atomic distances between two solutes.
For each encounter, the second solute is transformed and the minimum distance to `solute1_crds` is computed; pairwise absolute differences of these minima populate `matrix`.

Parameters

out	*matrix*	Output (n,n) matrix of pairwise values
out	*array*	Output (n) array with per-encounter minimum distances
in	*n*	Number of encounters
in	*nb_atoms*	Number of atoms in solute 2 to transform
in	*xc1,xc2*	Solute centers used for translation transformations
in	*trans_vector*	Transform translation vectors (n,3)
in	*rot1,rot2*	Transform rotation arrays (n,3)
in	*solute1_crds*	Coordinates of first solute atoms
in	*solute2_crds*	Coordinates of second solute atoms

9.2.1.3 array_z_coord()

```
subroutine mod_array::array_z_coord (
            type(type_assoc_file), intent(inout) complexes,
            real(kind=8), dimension(n), intent(out) array_sorted,
```

```
            integer, intent(in) n,
            integer, intent(in) nb_atoms,
            real(kind=8), dimension(3), intent(in) xc1,
            real(kind=8), dimension(3), intent(in) xc2,
            real(kind=8), dimension(:, :), intent(in) trans_vector,
            real(kind=8), dimension(:, :), intent(in) rot1,
            real(kind=8), dimension(:, :), intent(in) rot2,
            real(kind=8), dimension(:, :), intent(in) solute_crds )
```

Build a matrix of absolute differences of Z-coordinates per encounter.
Computes the Z coordinate for each encounter (as in `array_z_coord`) and fills `matrix(i,j) = |z_i - z_j|` for i<j (symmetrically).

Parameters

`out`	*array*	Output array with per-encounter Z values
`in`	*n*	Number of encounters
`in`	*nb_atoms*	Number of atoms in `solute_crds`
`in`	*xc1,xc2*	Solute centers used for translation transformations
`in`	*trans_vector,rot1,rot2*	Transform arrays (n,3)
`in`	*solute_crds*	Coordinates of solute atoms

9.2.1.4 merge_sorted_segments()

```
subroutine mod_array::merge_sorted_segments (
            real(kind=8), dimension(:), intent(inout) arr,
            type(type_assoc_file), intent(inout) complexes,
            real(kind=8), dimension(:), intent(inout) tmp_arr,
            character(len=217), dimension(:), intent(inout) tmp_comp,
            integer, intent(in) left,
            integer, intent(in) mid,
            integer, intent(in) right )
```

Merge two adjacent sorted segments of arrays in place.
Merges `arr(left:mid)` and `arr(mid+1:right)` into a single sorted segment `arr(left:right)`, reordering `comp` in the same way.

Parameters

`in,out`	*arr*	Array of values being sorted
`in,out`	*comp*	Companion array reordered alongside `arr`
`out`	*tmp_arr*	Temporary buffer for values (at least size `right`)
`out`	*tmp_comp*	Temporary buffer for companion (at least size `right`)
`in`	*left*	Start index of the first segment
`in`	*mid*	End index of the first segment
`in`	*right*	End index of the second segment

9.2.1.5 read_array()

```
subroutine mod_array::read_array (
            real(kind=8), dimension(:), intent(out), allocatable array,
            integer, intent(inout) n,
            character*128, intent(in) filename )
```

9.2.1.6 write_array()

```
subroutine mod_array::write_array (
            real(kind=8), dimension(:), intent(in) array,
            character*128, intent(in) filename )
```

9.3 mod_assoc Module Reference

Association/complexes file utilities.

Data Types

- type type_assoc_file

  Data type for storing binned data (eg.

Functions/Subroutines

- subroutine allocate_assoc_object (this, nlines)

  Allocate storage for an association/complexes object.
- integer function size_assoc (assoc_file)

  Count encounter records in an association/complexes file.
- subroutine fill_assoc_object (this, assoc_file)

  Read an association/complexes file and populate `type_assoc_file`.
- subroutine write_complexes (complexes, nb_encounters, complexes_filename)

9.3.1 Detailed Description

Association/complexes file utilities.
This module defines the `type_assoc_file` derived type and routines to allocate and populate it from an association/complexes text file. The `type_assoc_file` stores the raw file lines, header lines, per-encounter translation vectors and rotation vectors, and the two reference centers `xc1` and `xc2` extracted from the file header.
NOTE: The routines in this file are adapted from the SDA7 project's `mod_pdb.f90` (original author Neil Bruce). This file contains code and ideas derived from that implementation; appropriate attribution to the SDA7 sources is retained here. Consult the SDA7 project for original authorship and licensing details.

Author

Abraham Muñiz-Chicharro

9.3.2 Function/Subroutine Documentation

9.3.2.1 allocate_assoc_object()

```
subroutine mod_assoc::allocate_assoc_object (
            type(type_assoc_file), intent(inout) this,
            integer, intent(in) nlines )
```

Allocate storage for an association/complexes object.
Allocates the arrays inside `this` to hold `nlines` encounter records and initializes numeric arrays to zero.

Parameters

`in,out`	*this*	Instance of `type_assoc_file` to initialize
`in`	*nlines*	Number of encounter lines to allocate

9.3.2.2 fill_assoc_object()

```
subroutine mod_assoc::fill_assoc_object (
            type(type_assoc_file), intent(inout) this,
            integer, intent(in) assoc_file )
```

Read an association/complexes file and populate type_assoc_file.
Reads header lines from the open unit assoc_file and extracts the two reference centers (xc1, xc2) from header lines 3 and 4. Subsequent non-header lines are stored into thislines and the translation and rotation vectors are parsed into thistrans_vector, thisrot1 and thisrot2.

Parameters

in,out	*this*	Allocated type_assoc_file to populate
in	*assoc_file*	Fortran unit number of an already-open file

9.3.2.3 size_assoc()

```
integer function mod_assoc::size_assoc (
            integer assoc_file )
```

Count encounter records in an association/complexes file.
Scans assoc_file (a Fortran unit number) and counts the number of non-comment lines. Lines starting with # are treated as comments and ignored. Returns the number of encounter records.

Parameters

in	*assoc_file*	Fortran unit number of an already-open file

Returns

nlines Number of encounter records found

9.3.2.4 write_complexes()

```
subroutine mod_assoc::write_complexes (
            type(type_assoc_file), intent(in) complexes,
            integer, intent(in) nb_encounters,
            character(len=*), intent(in) complexes_filename )
```

9.4 mod_clust_algorithm Module Reference

Functions/Subroutines

- subroutine linkage_clustering_from_matrix (matrix, n, linkage_type, output_name, complexes, use_cuda)

  Perform hierarchical clustering on a distance matrix.
- subroutine linkage_clustering_from_array (array, n, linkage_type, output_name, complexes, use_cuda)

  Perform hierarchical clustering on a distance matrix.
- subroutine sort_complexes (tot_encounters, cluster_indexes, cluster_count, active_clusters, representative↩ _indexes, cluster_average, cluster_sd, array)

  Sort clusters and associated arrays by a given key array.
- subroutine write_cluster_elements (tot_encounters, cluster_parent, cluster_count, active_clusters, output_↩ name)

  Write cluster membership lists to a text file.

- subroutine write_clust_info (tot_encounters, representative_indexes, active_clusters, clust_count, cluster_↩ average, cluster_sd, output_name)

  Write summary information about clusters to a text file.
- subroutine write_cluster_complexes (tot_encounters, cluster_parent, cluster_count, active_clusters, output↩ _name, complexes)

  Write complexes contained in each cluster to individual files.

9.4.1 Function/Subroutine Documentation

9.4.1.1 linkage_clustering_from_array()

```
subroutine mod_clust_algorithm::linkage_clustering_from_array (
            real (kind=8), dimension(:), intent(inout), allocatable array,
            integer, intent(in) n,
            character(len=*), intent(in) linkage_type,
            character*128, intent(in) output_name,
            type(type_assoc_file) complexes,
            logical, intent(in), optional use_cuda )
```

Perform hierarchical clustering on a distance matrix.

This routine performs hierarchical clustering using minimum, maximum, or mean linkage on the input `matrix` (shape `(n,n)`) of pairwise distances or similarity scores. The routine mutates `matrix` during merging and terminates merging when clusters are farther apart than an internally computed threshold. Results are written to files named using `output_name` and complexes information is optionally recorded via the `complexes` structure.

Parameters

`in,out`	*matrix*	Pairwise distance/similarity matrix (n,n). This matrix is updated during clustering.
`in`	*n*	Number of elements / encounters (integer).
`in`	*linkage_type*	Type of linkage: 'min', 'max', or 'mean' (string).
`in`	*output_name*	Base name for output files (string).
`in`	*complexes*	Structure containing complex text lines/headers.
`in`	*opt_array*	Optional array of per-encounter values used to compute cluster averages instead of reading values from `matrix`.

9.4.1.2 linkage_clustering_from_matrix()

```
subroutine mod_clust_algorithm::linkage_clustering_from_matrix (
            real (kind=8), dimension(:, :), intent(inout), allocatable matrix,
            integer, intent(in) n,
            character(len=*), intent(in) linkage_type,
            character*128, intent(in) output_name,
            type(type_assoc_file) complexes,
            logical, intent(in), optional use_cuda )
```

Perform hierarchical clustering on a distance matrix.

This routine performs hierarchical clustering using minimum, maximum, or mean linkage on the input `matrix` (shape `(n,n)`) of pairwise distances or similarity scores. The routine mutates `matrix` during merging and terminates merging when clusters are farther apart than an internally computed threshold. Results are written to files named using `output_name` and complexes information is optionally recorded via the `complexes` structure.

Parameters

`in,out`	*matrix*	Pairwise distance/similarity matrix (n,n). This matrix is updated during clustering.
`in`	*n*	Number of elements / encounters (integer).

Parameters

in	*linkage_type*	Type of linkage: 'min', 'max', or 'mean' (string).
in	*output_name*	Base name for output files (string).
in	*complexes*	Structure containing complex text lines/headers.
in	*opt_array*	Optional array of per-encounter values used to compute cluster averages instead of reading values from `matrix`.

9.4.1.3 sort_complexes()

```
subroutine mod_clust_algorithm::sort_complexes (
            integer, intent(in) tot_encounters,
            integer, dimension(:, :), intent(inout) cluster_indexes,
            integer, dimension(:), intent(inout) cluster_count,
            logical, dimension(:), intent(inout) active_clusters,
            integer, dimension(:), intent(inout) representative_indexes,
            real (kind=8), dimension(:), intent(inout) cluster_average,
            real (kind=8), dimension(:), intent(inout) cluster_sd,
            real (kind=8), dimension(:), intent(inout) array )
```

Sort clusters and associated arrays by a given key array.
Reorders `cluster_indexes` and all supplied per-cluster arrays so that they are sorted by ascending values in `array`. This is a stable reordering helper used prior to writing results.

Parameters

in	*tot_encounters*	Number of clusters/encounters
in,out	*cluster_indexes*	Matrix storing cluster member indexes
in,out	*cluster_count*	Number of members in each cluster
in,out	*active_clusters*	Logical flags marking active clusters
in,out	*representative_indexes*	Representative index per cluster
in,out	*cluster_average*	Per-cluster average values
in,out	*cluster_sd*	Per-cluster standard deviation
in,out	*array*	Key array used to sort clusters

9.4.1.4 write_clust_info()

```
subroutine mod_clust_algorithm::write_clust_info (
            integer, intent(in) tot_encounters,
            integer, dimension(:), intent(in) representative_indexes,
            logical, dimension(:), intent(in) active_clusters,
            integer, dimension(tot_encounters), intent(in) clust_count,
            real (kind=8), dimension(tot_encounters) cluster_average,
            real (kind=8), dimension(tot_encounters) cluster_sd,
            character*128, intent(in) output_name )
```

Write summary information about clusters to a text file.
Produces a summary file `<output_name>_clust_info.txt` listing cluster population, representative index, average and standard deviation for each active cluster.

Parameters

in	*tot_encounters*	Total number of encounters
in	*clust_count*	Array with population of each cluster

Parameters

in	*active_clusters*	Logical mask indicating active clusters
in	*representative_indexes*	Representative index per cluster
in	*output_name*	Base filename for outputs
in	*cluster_average*	Per-cluster average values
in	*cluster_sd*	Per-cluster standard deviations

9.4.1.5 write_cluster_complexes()

```
subroutine mod_clust_algorithm::write_cluster_complexes (
            integer, intent(in) tot_encounters,
            integer, dimension(:), intent(in) cluster_parent,
            integer, dimension(:), intent(in) cluster_count,
            logical, dimension(:), intent(in) active_clusters,
            character*128, intent(in) output_name,
            type(type_assoc_file) complexes )
```

Write complexes contained in each cluster to individual files.
For each active cluster this routine creates a file `<output_name>_clusterXXXX_complexes` containing the header lines from `complexeshead` and the encounter lines for the cluster members taken from `complexeslines`.

Parameters

in	*tot_encounters*	Number of clusters/encounters
in	*cluster_indexes*	Matrix of cluster member indexes
in	*cluster_count*	Number of members per cluster
in	*active_clusters*	Logical mask indicating active clusters
in	*output_name*	Base filename for output files
in	*complexes*	Structure with header and lines to write

9.4.1.6 write_cluster_elements()

```
subroutine mod_clust_algorithm::write_cluster_elements (
            integer, intent(in) tot_encounters,
            integer, dimension(:), intent(in) cluster_parent,
            integer, dimension(:), intent(in) cluster_count,
            logical, dimension(:), intent(in) active_clusters,
            character*128, intent(in) output_name )
```

Write cluster membership lists to a text file.
Outputs a file named `<output_name>_clusters.txt` containing the members of each active cluster in a matrix-like format.

Parameters

in	*tot_encounters*	Number of possible clusters/encounters
in	*cluster_indexes*	Matrix of cluster member indexes
in	*cluster_count*	Number of members per cluster
in	*active_clusters*	Logical mask indicating active clusters
in	*output_name*	Base filename for output

9.5 mod_cuda Module Reference

Functions/Subroutines

- integer function, public cuda_matrix_clustering (matrix, n, dist_threshold, linkage_str, cluster_parent, cluster_count, active_points, cluster_size)

  Initialize GPU clustering - transfer matrix to GPU once.

9.5.1 Function/Subroutine Documentation

9.5.1.1 cuda_matrix_clustering()

```
integer function, public mod_cuda::cuda_matrix_clustering (
            real(kind=8), dimension(:, :), intent(in), target matrix,
            integer, intent(in) n,
            real(kind=8), intent(in) dist_threshold,
            character(len=*), intent(in) linkage_str,
            integer, dimension(:), intent(inout), target cluster_parent,
            integer, dimension(:), intent(inout), target cluster_count,
            logical, dimension(:), intent(inout) active_points,
            integer, dimension(:), intent(in), target cluster_size )
```

Initialize GPU clustering - transfer matrix to GPU once.

Parameters

in	*matrix*	Distance matrix (n x n, column-major)
in	*n*	Matrix dimension
in	*active_points*	Logical array of active clusters
in	*cluster_size*	Integer array of cluster sizes

Returns

0 on success, -1 on error Find minimum pair and merge on GPU

Parameters

out	*min_dist*	Minimum distance value
out	*min_↩ i,min_j*	Indices of minimum pair (1-based)
in	*linkage_str*	Linkage type ('min', 'max', or 'mean')

Returns

0 on success, -1 on error Perform complete GPU clustering - all iterations on GPU

Parameters

in	*matrix*	Distance matrix (n x n, column-major)
in	*n*	Matrix dimension
in	*dist_threshold*	Distance threshold for stopping
in	*linkage_str*	Linkage type ('min', 'max', or 'mean')
in,out	*cluster_parent*	Output cluster parent for each point (n) - O(n) tracking
in,out	*cluster_count*	Output cluster sizes (n)

Parameters

in,out	*active_points*	Output active clusters (n)
in,out	*cluster_size*	Cluster sizes for linkage computation

Returns

0 on success, -1 on error

9.6 mod_matrix Module Reference

Functions/Subroutines

- subroutine matrix_z_coord (matrix, array, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, solute_crds)

  Build a matrix of absolute differences of Z-coordinates per encounter.
- subroutine matrix_atoms_dist (matrix, array, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, solute1_crds, solute2_crds)

  Build matrix from minimum inter-atomic distances between two solutes.
- subroutine matrix_angle (matrix, array, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, point1a, point1b, point2a, point2b, dimensions)

  Build matrix of angular differences between encounters.
- subroutine matrix_rmsd (matrix, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, coord)

  Build RMSD-based pairwise matrix between transformed coordinates.
- subroutine write_matrix (matrix, filename)

  Write a numeric matrix to a formatted text file.
- subroutine read_matrix (matrix, n, filename)

  Write a numeric array to a formatted text file.

9.6.1 Function/Subroutine Documentation

9.6.1.1 matrix_angle()

```
subroutine mod_matrix::matrix_angle (
             real(kind=8), dimension(n, n), intent(out) matrix,
             real(kind=8), dimension(n), intent(out) array,
             integer, intent(in) n,
             integer, intent(in) nb_atoms,
             real(kind=8), dimension(3), intent(in) xc1,
             real(kind=8), dimension(3), intent(in) xc2,
             real(kind=8), dimension(:, :), intent(in) trans_vector,
             real(kind=8), dimension(:, :), intent(in) rot1,
             real(kind=8), dimension(:, :), intent(in) rot2,
             real(kind=8), dimension(:), intent(in) point1a,
             real(kind=8), dimension(:), intent(in) point1b,
             real(kind=8), dimension(:), intent(in) point2a,
             real(kind=8), dimension(:), intent(in) point2b,
             integer, intent(in) dimensions )
```

Build matrix of angular differences between encounters.
Computes the angle (2D or 3D) for each encounter and fills the pairwise absolute difference matrix.

Parameters

out	*matrix*	Output (n,n) matrix of pairwise angular differences
out	*array*	Output (n) array with per-encounter angles

Parameters

in	*n*	Number of encounters
in	*nb_atoms*	Number of atoms in solute 2 to transform
in	*xc1,xc2*	Solute centers used for translation transformations
in	*trans_vector*	Transform translation vectors (n,3)
in	*rot1,rot2*	Transform rotation arrays (n,3)
in	*point1a*	First point defining reference vector
in	*point1b*	Second point defining reference vector
in	*point2a*	First point of vector to be transformed
in	*point2b*	Second point of vector to be transformed
in	*dimensions*	Dimensionality of angle calculation (2 or 3)

9.6.1.2 matrix_atoms_dist()

```
subroutine mod_matrix::matrix_atoms_dist (
             real(kind=8), dimension(n, n), intent(out) matrix,
             real(kind=8), dimension(n), intent(out) array,
             integer, intent(in) n,
             integer, intent(in) nb_atoms,
             real(kind=8), dimension(3), intent(in) xc1,
             real(kind=8), dimension(3), intent(in) xc2,
             real(kind=8), dimension(:, :), intent(in) trans_vector,
             real(kind=8), dimension(:, :), intent(in) rot1,
             real(kind=8), dimension(:, :), intent(in) rot2,
             real(kind=8), dimension(:, :), intent(in) solute1_crds,
             real(kind=8), dimension(:, :), intent(in) solute2_crds )
```

Build matrix from minimum inter-atomic distances between two solutes.
For each encounter, the second solute is transformed and the minimum distance to `solute1_crds` is computed; pairwise absolute differences of these minima populate `matrix`.

Parameters

out	*matrix*	Output (n,n) matrix of pairwise values
out	*array*	Output (n) array with per-encounter minimum distances
in	*n*	Number of encounters
in	*nb_atoms*	Number of atoms in solute 2 to transform
in	*xc1,xc2*	Solute centers used for translation transformations
in	*trans_vector*	Transform translation vectors (n,3)
in	*rot1,rot2*	Transform rotation arrays (n,3)
in	*solute1_crds*	Coordinates of first solute atoms
in	*solute2_crds*	Coordinates of second solute atoms

9.6.1.3 matrix_rmsd()

```
subroutine mod_matrix::matrix_rmsd (
             real(kind=8), dimension(n, n), intent(out) matrix,
             integer, intent(in) n,
             integer, intent(in) nb_atoms,
             real(kind=8), dimension(3), intent(in) xc1,
             real(kind=8), dimension(3), intent(in) xc2,
```

```
            real(kind=8), dimension(:, :), intent(in) trans_vector,
            real(kind=8), dimension(:, :), intent(in) rot1,
            real(kind=8), dimension(:, :), intent(in) rot2,
            real(kind=8), dimension(:, :), intent(in) coord )
```

Build RMSD-based pairwise matrix between transformed coordinates.
Applies transformation to the coordinate set for each encounter and computes pairwise RMSD values between transformed coordinate sets.

Parameters

out	*matrix*	Output (n,n) matrix of pairwise RMSD values
in	*n*	Number of encounters
in	*nb_atoms*	Number of atoms in solute 2 to transform
in	*xc1,xc2*	Solute centers used for translation transformations
in	*trans_vector*	Transform translation vectors (n,3)
in	*rot1,rot2*	Transform rotation arrays (n,3)
in	*coord*	Coordinates to be transformed

9.6.1.4 matrix_z_coord()

```
subroutine mod_matrix::matrix_z_coord (
            real(kind=8), dimension(n, n), intent(out) matrix,
            real(kind=8), dimension(n), intent(out) array,
            integer, intent(in) n,
            integer, intent(in) nb_atoms,
            real(kind=8), dimension(3), intent(in) xc1,
            real(kind=8), dimension(3), intent(in) xc2,
            real(kind=8), dimension(:, :), intent(in) trans_vector,
            real(kind=8), dimension(:, :), intent(in) rot1,
            real(kind=8), dimension(:, :), intent(in) rot2,
            real(kind=8), dimension(:, :), intent(in) solute_crds )
```

Build a matrix of absolute differences of Z-coordinates per encounter.
Computes the Z coordinate for each encounter (as in `array_z_coord`) and fills `matrix(i,j) =` $|$`z_i - z_j`$|$ for i$<$j (symmetrically).

Parameters

out	*matrix*	Output (n,n) matrix of pairwise values
out	*array*	Output array with per-encounter Z values
in	*n*	Number of encounters
in	*nb_atoms*	Number of atoms in `solute_crds`
in	*xc1,xc2*	Solute centers used for translation transformations
in	*trans_vector,rot1,rot2*	Transform arrays (n,3)
in	*solute_crds*	Coordinates of solute atoms

9.6.1.5 read_matrix()

```
subroutine mod_matrix::read_matrix (
            real(kind=8), dimension(:, :), intent(out), allocatable matrix,
            integer, intent(inout) n,
            character*128, intent(in) filename )
```

Write a numeric array to a formatted text file.

Writes `array` as a single formatted line using `F10.4` floats.

Parameters

`in`	*array*	Input array to write
`in`	*filename*	Output filename Read a square numeric matrix from a formatted text file.

Reads the file line-by-line to determine matrix size and parses rows with the expected `F10.4` formatting.

Parameters

`out`	*matrix*	Output matrix read from file (allocated)
`in,out`	*n*	Number of encounters (adjusted if file is smaller)
`in`	*filename*	Input filename

9.6.1.6 write_matrix()

```
subroutine mod_matrix::write_matrix (
            real(kind=8), dimension(:, :), intent(in) matrix,
            character*128, intent(in) filename )
```

Write a numeric matrix to a formatted text file.
Writes `matrix` with each row as `F10.4` formatted floats. The output filename is provided by `filename`.

Parameters

`in`	*matrix*	Input matrix to write
`in`	*filename*	Output filename

9.7 mod_memory Module Reference

Functions/Subroutines

- integer(kind=8) function, public get_available_memory ()

  Utilities for memory management and batch processing.
- integer function, public calculate_max_batch_size (available_memory, nb_total_encounters, reserve_↩ percent)

  Calculate maximum batch size for matrix processing.

9.7.1 Function/Subroutine Documentation

9.7.1.1 calculate_max_batch_size()

```
integer function, public mod_memory::calculate_max_batch_size (
            integer(kind=8), intent(in) available_memory,
            integer, intent(in) nb_total_encounters,
            integer, intent(in), optional reserve_percent )
```

Calculate maximum batch size for matrix processing.
Given available memory and memory to leave free (in percentage), calculates the maximum number of encounters that can be processed.
Formula: usable_memory = available_memory $*$ (1 - reserve_percent/100) bytes_per_encounter = nb_total_↩ encounters $*$ 8 + other_overhead max_batch = usable_memory / bytes_per_encounter

Parameters

in	*available_memory*	Available system RAM in bytes
in	*nb_total_encounters*	Total number of encounters to process
in	*reserve_percent*	Percentage of memory to keep free (default 20)

Returns

maximum batch size (number of encounters per batch)

9.7.1.2 get_available_memory()

`integer(kind=8) function, public mod_memory::get_available_memory`

Utilities for memory management and batch processing.
This module provides routines to detect available system memory, calculate batch sizes for large datasets, and manage memory-efficient processing of large matrices.

Author

Abraham Muñiz-Chicharro Get available system memory in bytes

Detects available RAM on Linux systems by parsing /proc/meminfo. On other systems, returns -1.

Returns

available memory in bytes, or -1 if cannot be determined

9.8 mod_pdb Module Reference

Module to define a pdb type and pdb reading routine for tools.

Data Types

- type type_pdb_atom
- type type_pdb_chain
- type type_pdb_file

  Data type for storing binned data (eg.
- type type_pdb_residue

Functions/Subroutines

- subroutine allocate_pdb_object (this, pdb_file, natoms, nresidues, nchains)

  Allocate and initialize a `type_pdb_file` structure from a PDB file.
- integer function count_atoms (pdb_file)

  Count ATOM/HETATM records in a PDB file.
- integer function count_residues (pdb_file)

  Count residues in a PDB file.
- integer function count_chains (pdb_file)

  Count chain separators (TER records) in a PDB file.
- subroutine fill_pdb_object (this, pdb_file)

  Parse PDB records and populate the `type_pdb_file` structure.

9.8.1 Detailed Description

Module to define a pdb type and pdb reading routine for tools.
NOTE: This implementation is based on the SDA7 project's mod_pdb.f90 (original author Neil Bruce). The present file has been modified and expanded for the clustering tools in this workspace. Notable enhancements include the addition of count_residues and count_chains functions and changes that allow creation of richer in-memory objects where residues contain their atoms and chains contain residues and atoms. These changes make it possible to build type_pdb_file instances with proper residues and chains substructures (each storing their atoms) rather than storing atoms in a single flat list only.
The original SDA7 code and author attribution are retained; consult the SDA7 sources for original licensing and authorship details.

Author

Neil Bruce @modified-by Abraham Muñiz-Chicharro

9.8.2 Function/Subroutine Documentation

9.8.2.1 allocate_pdb_object()

```
subroutine mod_pdb::allocate_pdb_object (
            type(type_pdb_file), intent(inout) this,
            integer, intent(in) pdb_file,
            integer, intent(in) natoms,
            integer, intent(in) nresidues,
            integer, intent(in) nchains )
```

Allocate and initialize a type_pdb_file structure from a PDB file.
This routine allocates the storage for thisatoms, thisresidues and thischains using the supplied counts and performs a preliminary scan of the open Fortran unit pdb_file to determine per-chain and per-residue allocation sizes for nested arrays.

Parameters

in,out	*this*	type_pdb_file instance to allocate and initialize
in	*pdb_file*	Fortran unit number of an already-open PDB file
in	*natoms*	Total number of ATOM/HETATM records expected
in	*nresidues*	Number of residues expected
in	*nchains*	Number of chains expected

9.8.2.2 count_atoms()

```
integer function mod_pdb::count_atoms (
            integer pdb_file )
```

Count ATOM/HETATM records in a PDB file.
Scans the open Fortran file unit pdb_file and returns the number of lines starting with ATOM or HETATM.

Parameters

in	*pdb_file*	Fortran unit number of an already-open PDB file

Returns

natoms Number of ATOM/HETATM records found

9.8.2.3 count_chains()

```
integer function mod_pdb::count_chains (
            integer pdb_file )
```

Count chain separators (TER records) in a PDB file.
Scans the open Fortran file unit `pdb_file` and counts the number of `TER` records, which are treated as chain separators. Returns the number of chains found.

Parameters

in	*pdb_file*	Fortran unit number of an already-open PDB file

Returns

nchains Number of chains (TER records) found

9.8.2.4 count_residues()

```
integer function mod_pdb::count_residues (
            integer pdb_file )
```

Count residues in a PDB file.
Scans the open Fortran file unit `pdb_file` and counts residue changes by comparing residue identifiers in AT↩ OM/HETATM records. Returns the number of distinct residues found.

Parameters

in	*pdb_file*	Fortran unit number of an already-open PDB file

Returns

nresidues Number of residues found

9.8.2.5 fill_pdb_object()

```
subroutine mod_pdb::fill_pdb_object (
            type(type_pdb_file), intent(inout) this,
            integer, intent(in) pdb_file )
```

Parse PDB records and populate the `type_pdb_file` structure.
Reads ATOM/HETATM records from the already-open Fortran unit `pdb_file`, fills `thisatoms` with atom records and populates `thisresidues` and `thischains` so that residues contain their per-residue `atoms` arrays and chains contain their residues and atoms. The routine expects `this` to have been allocated with the correct sizes (see `allocate_pdb_object`).

Parameters

in,out	*this*	Pre-allocated `type_pdb_file` to populate
in	*pdb_file*	Fortran unit number of an already-open PDB file

9.9 mod_threshold Module Reference

Functions/Subroutines

- subroutine array_z_coord (array, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, solute_crds)

Compute the Z-coordinate (along transformed z-axis) for each encounter.

- subroutine array_atoms_dist (array, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, solute1_crds, solute2_↩ crds)

  Compute minimum inter-atomic distance between two solutes for each encounter.

- subroutine array_angle (array, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, point1a, point1b, point2a, point2b, dimensions)

  Compute angle between two vectors (defined by atom groups) per encounter.

- subroutine sort_array (arr, encounter_indexes)

  Sort an array in ascending order and reorder encounter indexes accordingly.

9.9.1 Function/Subroutine Documentation

9.9.1.1 array_angle()

```
subroutine mod_threshold::array_angle (
             real(kind=8), dimension(n), intent(out) array,
             integer, intent(in) n,
             integer, intent(in) nb_atoms,
             real(kind=8), dimension(3), intent(in) xc1,
             real(kind=8), dimension(3), intent(in) xc2,
             real(kind=8), dimension(:, :), intent(in) trans_vector,
             real(kind=8), dimension(:, :), intent(in) rot1,
             real(kind=8), dimension(:, :), intent(in) rot2,
             real(kind=8), dimension(:), intent(in) point1a,
             real(kind=8), dimension(:), intent(in) point1b,
             real(kind=8), dimension(:), intent(in) point2a,
             real(kind=8), dimension(:), intent(in) point2b,
             integer, intent(in) dimensions )
```

Compute angle between two vectors (defined by atom groups) per encounter.
Transforms the pair of points from solute2 for each encounter and computes the angle between the vector defined by `point1b-point1a` and the vector defined by the transformed `point2b-point2a`.

Parameters

`out`	*array*	Output array with angles in degrees (n)
`in`	*n*	Number of encounters
`in`	*nb_atoms*	Number of atoms used for transformation
`in`	*xc1,xc2*	Centers used for transform
`in`	*trans_vector,rot1,rot2*	Transform arrays (n,3)
`in`	*point1a,b,point2a,b*	Reference points/vectors
`in`	*dimensions*	Use 2 or 3 to select projection

9.9.1.2 array_atoms_dist()

```
subroutine mod_threshold::array_atoms_dist (
             real(kind=8), dimension(n), intent(out) array,
             integer, intent(in) n,
             integer, intent(in) nb_atoms,
             real(kind=8), dimension(3), intent(in) xc1,
             real(kind=8), dimension(3), intent(in) xc2,
             real(kind=8), dimension(:, :), intent(in) trans_vector,
             real(kind=8), dimension(:, :), intent(in) rot1,
```

```
            real(kind=8), dimension(:, :), intent(in) rot2,
            real(kind=8), dimension(:, :), intent(in) solute1_crds,
            real(kind=8), dimension(:, :), intent(in) solute2_crds )
```

Compute minimum inter-atomic distance between two solutes for each encounter.
For each encounter the second solute is transformed and the minimum distance between any atom of `solute1_crds` and the transformed `solute2_crds` is computed and stored in `array`.

Parameters

`out`	*array*	Output array (n)
`in`	*n*	Number of encounters
`in`	*nb_atoms*	Number of atoms in `solute2_crds`
`in`	*xc1,xc2*	Centers used for transform
`in`	*trans_vector,rot1,rot2*	Transform arrays (n,3)
`in`	*solute1_crds*	Coordinates of solute1 atoms
`in`	*solute2_crds*	Coordinates of solute2 atoms

9.9.1.3 array_z_coord()

```
subroutine mod_threshold::array_z_coord (
            real(kind=8), dimension(n), intent(out) array,
            integer, intent(in) n,
            integer, intent(in) nb_atoms,
            real(kind=8), dimension(3), intent(in) xc1,
            real(kind=8), dimension(3), intent(in) xc2,
            real(kind=8), dimension(:, :), intent(in) trans_vector,
            real(kind=8), dimension(:, :), intent(in) rot1,
            real(kind=8), dimension(:, :), intent(in) rot2,
            real(kind=8), dimension(:, :), intent(in) solute_crds )
```

Compute the Z-coordinate (along transformed z-axis) for each encounter.
Recenters each encounter at `xc2`, applies rotation/translation using `rot1/rot2` and `trans_vector`, and extracts the Z coordinate of the first atom in `solute_crds` for every encounter into `array`.

Parameters

`out`	*array*	Output array of length `n` with z-coordinates
`in`	*n*	Number of encounters
`in`	*nb_atoms*	Number of atoms in `solute_crds`
`in`	*xc1,xc2*	Centers used for transform
`in`	*trans_vector,rot1,rot2*	Transform arrays (n,3)
`in`	*solute_crds*	Coordinates of solute atoms (nb_atoms,3)

9.9.1.4 sort_array()

```
subroutine mod_threshold::sort_array (
            real (kind=8), dimension(:), intent(inout) arr,
            integer, dimension(:), intent(inout) encounter_indexes )
```

Sort an array in ascending order and reorder encounter indexes accordingly.
Simple selection sort used to reorder `arr` and the parallel `encounter_indexes` array. Intended for small to moderate sizes.

Parameters

in,out	*arr*	Array to sort (modified in-place)
in,out	*encounter_indexes*	Parallel index array reordered to match `arr`

9.10 read_input Module Reference

Functions/Subroutines

- subroutine read_pdb (pdb, filename, tot_atoms, tot_residues, tot_chains)

  Read a PDB file into a `type_pdb_file` object.
- subroutine read_assoc (assoc, filename, tot_encounters)

  Read an association/complexes file into a `type_assoc_file` object.

9.10.1 Function/Subroutine Documentation

9.10.1.1 read_assoc()

```
subroutine read_input::read_assoc (
            type ( type_assoc_file ) assoc,
            character*128, intent(in) filename,
            integer, intent(out) tot_encounters )
```

Read an association/complexes file into a `type_assoc_file` object.
Checks the file exists, opens it and uses `size_assoc`, `allocate_assoc_object` and `fill_assoc_↩object` to construct the `assoc` structure.

Parameters

out	*assoc*	Allocated and filled assoc object
in	*filename*	Path to complexes file
out	*tot_encounters*	Number of encounter records read

9.10.1.2 read_pdb()

```
subroutine read_input::read_pdb (
            type ( type_pdb_file ) pdb,
            character*128, intent(in) filename,
            integer, intent(out) tot_atoms,
            integer, intent(out) tot_residues,
            integer, intent(out) tot_chains )
```

Read a PDB file into a `type_pdb_file` object.
Validates the existence of `filename`, opens it, determines the counts for atoms, residues and chains, allocates `pdb` and fills it using `allocate_pdb_object` and `fill_pdb_object`.

Parameters

out	*pdb*	Allocated and filled PDB object
in	*filename*	Path to PDB file
out	*tot_atoms*	Number of atoms found
out	*tot_residues*	Number of residues found
out	*tot_chains*	Number of chains found

Chapter 10

Data Type Documentation

10.1 mod_assoc::type_assoc_file Type Reference

Data type for storing binned data (eg.

Public Attributes

- integer nlines

  Number of lines in assoc_file file.
- character(len=217), dimension(:), allocatable lines
- character(len=217), dimension(4) head
- real(kind=8), dimension(:, :), allocatable trans_vector

  Translational vector.
- real(kind=8), dimension(:,:), allocatable rot1

  rotX, rotY
- real(kind=8), dimension(:,:), allocatable rot2
- real(kind=8), dimension(3) xc1

  center of mass of p2
- real(kind=8), dimension(3) xc2

10.1.1 Detailed Description

Data type for storing binned data (eg.
Radial distribution functions)

10.1.2 Member Data Documentation

10.1.2.1 head

```
character(len=217), dimension(4) mod_assoc::type_assoc_file::head
```

10.1.2.2 lines

```
character(len=217), dimension(:), allocatable mod_assoc::type_assoc_file::lines
```

10.1.2.3 nlines

```
integer mod_assoc::type_assoc_file::nlines
```

Number of lines in assoc_file file.

10.1.2.4 rot1

```
real (kind=8), dimension (:,:), allocatable mod_assoc::type_assoc_file::rot1
```
rotX, rotY

10.1.2.5 rot2

```
real (kind=8), dimension (:,:), allocatable mod_assoc::type_assoc_file::rot2
```

10.1.2.6 trans_vector

```
real (kind=8), dimension (:, :), allocatable mod_assoc::type_assoc_file::trans_vector
```
Translational vector.

10.1.2.7 xc1

```
real (kind=8), dimension (3) mod_assoc::type_assoc_file::xc1
```
center of mass of p2

10.1.2.8 xc2

```
real (kind=8), dimension (3) mod_assoc::type_assoc_file::xc2
```
The documentation for this type was generated from the following file:

- mod_assoc.f90

10.2 mod_pdb::type_pdb_atom Type Reference

Public Attributes

- integer record

  Number of atoms in pdb file Flag for record type, 1 = ATOM, 2 = HETATM.
- real(kind=8), dimension(3) coord

  Position, occupancy and temp factor - can be used to store other data eg in pqr.
- real(kind=8) occ
- real(kind=8) beta
- character(len=4) name

  Atom name, res name and chain name.
- character(len=3) resname
- character(len=1) chainid
- integer id

  Atom and residue numbers.
- integer resid

10.2.1 Member Data Documentation

10.2.1.1 beta

```
real (kind=8) mod_pdb::type_pdb_atom::beta
```

10.2.1.2 chainid

```
character (len=1) mod_pdb::type_pdb_atom::chainid
```

10.2.1.3 coord

```
real (kind=8), dimension (3) mod_pdb::type_pdb_atom::coord
```
Position, occupancy and temp factor - can be used to store other data eg in pqr.

10.2.1.4 id

```
integer mod_pdb::type_pdb_atom::id
```
Atom and residue numbers.

10.2.1.5 name

```
character (len=4) mod_pdb::type_pdb_atom::name
```
Atom name, res name and chain name.

10.2.1.6 occ

```
real (kind=8) mod_pdb::type_pdb_atom::occ
```

10.2.1.7 record

```
integer mod_pdb::type_pdb_atom::record
```
Number of atoms in pdb file Flag for record type, 1 = ATOM, 2 = HETATM.

10.2.1.8 resid

```
integer mod_pdb::type_pdb_atom::resid
```

10.2.1.9 resname

```
character (len=3) mod_pdb::type_pdb_atom::resname
```
The documentation for this type was generated from the following file:

- mod_pdb.f90

10.3 mod_pdb::type_pdb_chain Type Reference

Collaboration diagram for mod_pdb::type_pdb_chain:

10.4 mod_pdb::type_pdb_file Type Reference

Data type for storing binned data (eg.
Collaboration diagram for mod_pdb::type_pdb_file:

Public Attributes

- integer natoms

  Number of atoms in pdb file.
- integer nresidues
- integer nchains
- type(type_pdb_atom), dimension(:), allocatable atoms

  Flag for record type, 1 = ATOM, 2 = HETATM.
- type(type_pdb_residue), dimension(:), allocatable residues
- type(type_pdb_chain), dimension(:), allocatable chains

10.4.1 Detailed Description

Data type for storing binned data (eg.
Radial distribution functions)

10.4.2 Member Data Documentation

10.4.2.1 atoms

```
type(type_pdb_atom), dimension (:), allocatable mod_pdb::type_pdb_file::atoms
```

Flag for record type, 1 = ATOM, 2 = HETATM.

10.4.2.2 chains

```
type(type_pdb_chain), dimension (:), allocatable mod_pdb::type_pdb_file::chains
```

10.4.2.3 natoms

```
integer mod_pdb::type_pdb_file::natoms
```

Number of atoms in pdb file.

10.4.2.4 nchains

```
integer mod_pdb::type_pdb_file::nchains
```

10.4.2.5 nresidues

```
integer mod_pdb::type_pdb_file::nresidues
```

10.4.2.6 residues

```
type(type_pdb_residue), dimension (:), allocatable mod_pdb::type_pdb_file::residues
```

The documentation for this type was generated from the following file:

- mod_pdb.f90

10.5 mod_pdb::type_pdb_residue Type Reference

Collaboration diagram for mod_pdb::type_pdb_residue:

Public Attributes

- integer natoms

  Number of atoms in pdb file.
- character chainid

  Chain id.
- character(len=3) resname

  Residue name.
- integer resid

  Residue number.
- type(type_pdb_atom), dimension(:), allocatable atoms

  Flag for record type, 1 = ATOM, 2 = HETATM.

10.5.1 Member Data Documentation

10.5.1.1 atoms

```
type(type_pdb_atom), dimension (:), allocatable mod_pdb::type_pdb_residue::atoms
```

Flag for record type, 1 = ATOM, 2 = HETATM.

10.5.1.2 chainid

```
character mod_pdb::type_pdb_residue::chainid
```

Chain id.

10.5.1.3 natoms

```
integer mod_pdb::type_pdb_residue::natoms
```

Number of atoms in pdb file.

10.5.1.4 resid

```
integer mod_pdb::type_pdb_residue::resid
```

Residue number.

10.5.1.5 resname

```
character (len=3) mod_pdb::type_pdb_residue::resname
```

Residue name.

The documentation for this type was generated from the following file:

- mod_pdb.f90

Chapter 11

File Documentation

11.1 analyze_residues.f90 File Reference

Functions/Subroutines

- program main

  Analyze which residues are close to the surface across encounters.
- subroutine print_help ()

  Print usage/help text for `analyze_residues`.

11.1.1 Function/Subroutine Documentation

11.1.1.1 main()

`program main`
Analyze which residues are close to the surface across encounters.
This program reads a PDB for solute2 and an associations/complexes file, transforms residue centers for each encounter and counts how many encounters bring residues within a distance threshold from the surface. Results are written to `closest_residues.txt`.

Author

Abraham Muñiz-Chicharro

11.1.1.2 print_help()

`subroutine main::print_help`
Print usage/help text for `analyze_residues`.
Shows required command-line options and examples for running the residue analysis tool.

11.2 clust.f90 File Reference

Functions/Subroutines

- program main

  Command-line program to perform hierarchical clustering on encounter matrices.
- subroutine print_help ()

  Print usage and help information for the `clust` program.

11.2.1 Function/Subroutine Documentation

11.2.1.1 main()

`program main`
Command-line program to perform hierarchical clustering on encounter matrices.
Reads a pairwise matrix (and optional array) and runs hierarchical clustering with user-specified linkage method (min, max, or mean). Results are written to files using the `-output_name` base name. See `print_help` below for usage.

Author

Abraham Muñiz-Chicharro

11.2.1.2 print_help()

`subroutine main::print_help`
Print usage and help information for the `clust` program.
Describes command-line options and provides an example invocation.

11.3 clust_all.f90 File Reference

Functions/Subroutines

- program clust_all

  Command-line utility to build matrices and perform clustering in memory.
- subroutine print_help (help_option)

  Print usage/help text for `clust_all`.
- subroutine read_atoms_coord (atoms_array, atoms_coord, tot_atoms, pdb, pdb_filename)

  Read coordinates for a specified list of atoms (names or indices).

11.3.1 Function/Subroutine Documentation

11.3.1.1 clust_all()

`program clust_all`
Command-line utility to build matrices and perform clustering in memory.
This program combines matrix generation and clustering without writing the matrix to disk. Useful for large matrices that would exceed disk space.

Author

Abraham Muñiz-Chicharro

11.3.1.2 print_help()

```
subroutine clust_all::print_help (
            character(len=*), intent(in) help_option )
```

Print usage/help text for `clust_all`.

11.3.1.3 read_atoms_coord()

```
subroutine clust_all::read_atoms_coord (
            character(len=*), dimension(:), intent(in) atoms_array,
            real(kind=8), dimension(:, :), intent(out), allocatable atoms_coord,
            integer, intent(in) tot_atoms,
            type ( type_pdb_file ), intent(in) pdb,
            character(len=*), intent(in) pdb_filename )
```

Read coordinates for a specified list of atoms (names or indices).

11.4 make_data.f90 File Reference

Functions/Subroutines

- program main

  Command-line utility to build encounter-based matrices.
- subroutine print_help (help_option)

  Print usage/help text for `make_data`.
- subroutine read_atoms_coord (atoms_array, atoms_coord, tot_atoms, pdb, pdb_filename)

  Read coordinates for a specified list of atoms (names or indices).

11.4.1 Function/Subroutine Documentation

11.4.1.1 main()

```
program main
```

Command-line utility to build encounter-based matrices.
Usage: see `print_help` below. This program reads PDB(s) and an association/complex file and generates matrices (rmsd, z_coord, atoms_dist, angle) and arrays derived from encounter transforms.

Author

Abraham Muñiz-Chicharro

11.4.1.2 print_help()

```
subroutine main::print_help (
            character(len=*), intent(in) help_option )
```

Print usage/help text for `make_data`.
Provides short usage examples for the main program and for each supported matrix type.

11.4.1.3 read_atoms_coord()

```
subroutine main::read_atoms_coord (
            character(len=*), dimension(:), intent(in) atoms_array,
            real(kind=8), dimension(:, :), intent(out), allocatable atoms_coord,
            integer, intent(in) tot_atoms,
            type ( type_pdb_file ), intent(in) pdb,
            character(len=*), intent(in) pdb_filename )
```

Read coordinates for a specified list of atoms (names or indices).
Accepts either atom indices or atom names. If atom names are provided, all matching atoms in the PDB are returned (in order).

Parameters

in	*atoms_array*	Array of atom name strings or index strings

Parameters

in	*pdb_filename*	Reference filename for warnings
in	*tot_atoms*	Total number of atoms in pdb
out	*atoms_coord*	Allocated array with atom coordinates (N,3)

11.5 maths.f90 File Reference

Modules

- module maths

  Mathematical utilities and coordinate transformation routines.

Functions/Subroutines

- subroutine maths::cross (v1, v2, v3)

  Compute the cross product of two 3D vectors.
- subroutine maths::update_complex (xc1, xc2, trans, rot_vx, rot_vy, nb_atoms, coords, new_coords)

  Apply translation and rotation to a set of atomic coordinates.
- subroutine maths::vectors_angle_3d (vector1, vector2, theta_degrees)

  Compute the angle in degrees between two 3D vectors.
- subroutine maths::vectors_angle_2d (vector1, vector2, theta_degrees)

  Compute the angle in degrees between two vectors projected to 2D.
- subroutine maths::rmsd (rmsd_value, nb_atoms, coords1, coords2)

  Compute the root-mean-square deviation (RMSD) between two sets of coordinates.
- subroutine maths::calculate_cog (cog, coords, nb_atoms)

  Compute the center of geometry (centroid) of a set of points.
- subroutine maths::calculate_distance (p1, p2, dist)

  Compute Euclidean distance between two 3D points.

11.6 mod_array.f90 File Reference

Modules

- module mod_array

Functions/Subroutines

- subroutine mod_array::array_z_coord (complexes, array_sorted, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, solute_crds)

  Build a matrix of absolute differences of Z-coordinates per encounter.
- subroutine mod_array::array_atoms_dist (complexes, array_sorted, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, solute1_crds, solute2_crds)

  Build matrix from minimum inter-atomic distances between two solutes.
- subroutine mod_array::array_angle (complexes, array_sorted, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, point1a, point1b, point2a, point2b, dimensions)

  Build matrix of angular differences between encounters.
- subroutine mod_array::write_array (array, filename)
- subroutine mod_array::read_array (array, n, filename)
- subroutine mod_array::merge_sorted_segments (arr, complexes, tmp_arr, tmp_comp, left, mid, right)

  Merge two adjacent sorted segments of arrays in place.

11.7 mod_assoc.f90 File Reference

Data Types

- type mod_assoc::type_assoc_file

  Data type for storing binned data (eg.

Modules

- module mod_assoc

  Association/complexes file utilities.

Functions/Subroutines

- subroutine mod_assoc::allocate_assoc_object (this, nlines)

  Allocate storage for an association/complexes object.
- integer function mod_assoc::size_assoc (assoc_file)

  Count encounter records in an association/complexes file.
- subroutine mod_assoc::fill_assoc_object (this, assoc_file)

  Read an association/complexes file and populate `type_assoc_file`.
- subroutine mod_assoc::write_complexes (complexes, nb_encounters, complexes_filename)

11.8 mod_clust_algorithm.f90 File Reference

Modules

- module mod_clust_algorithm

Functions/Subroutines

- subroutine mod_clust_algorithm::linkage_clustering_from_matrix (matrix, n, linkage_type, output_name, complexes, use_cuda)

  Perform hierarchical clustering on a distance matrix.
- subroutine mod_clust_algorithm::linkage_clustering_from_array (array, n, linkage_type, output_name, complexes, use_cuda)

  Perform hierarchical clustering on a distance matrix.
- subroutine mod_clust_algorithm::sort_complexes (tot_encounters, cluster_indexes, cluster_count, active_↩ clusters, representative_indexes, cluster_average, cluster_sd, array)

  Sort clusters and associated arrays by a given key array.
- subroutine mod_clust_algorithm::write_cluster_elements (tot_encounters, cluster_parent, cluster_count, active_clusters, output_name)

  Write cluster membership lists to a text file.
- subroutine mod_clust_algorithm::write_clust_info (tot_encounters, representative_indexes, active_clusters, clust_count, cluster_average, cluster_sd, output_name)

  Write summary information about clusters to a text file.
- subroutine mod_clust_algorithm::write_cluster_complexes (tot_encounters, cluster_parent, cluster_count, active_clusters, output_name, complexes)

  Write complexes contained in each cluster to individual files.

11.9 mod_cuda.f90 File Reference

Modules

- module mod_cuda

Functions/Subroutines

- integer function, public mod_cuda::cuda_matrix_clustering (matrix, n, dist_threshold, linkage_str, cluster_↩ parent, cluster_count, active_points, cluster_size)

  Initialize GPU clustering - transfer matrix to GPU once.

11.10 mod_matrix.f90 File Reference

Modules

- module mod_matrix

Functions/Subroutines

- subroutine mod_matrix::matrix_z_coord (matrix, array, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, solute_crds)

  Build a matrix of absolute differences of Z-coordinates per encounter.
- subroutine mod_matrix::matrix_atoms_dist (matrix, array, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, solute1_crds, solute2_crds)

  Build matrix from minimum inter-atomic distances between two solutes.
- subroutine mod_matrix::matrix_angle (matrix, array, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, point1a, point1b, point2a, point2b, dimensions)

  Build matrix of angular differences between encounters.
- subroutine mod_matrix::matrix_rmsd (matrix, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, coord)

  Build RMSD-based pairwise matrix between transformed coordinates.
- subroutine mod_matrix::write_matrix (matrix, filename)

  Write a numeric matrix to a formatted text file.
- subroutine mod_matrix::read_matrix (matrix, n, filename)

  Write a numeric array to a formatted text file.

11.11 mod_memory.f90 File Reference

Modules

- module mod_memory

Functions/Subroutines

- integer(kind=8) function, public mod_memory::get_available_memory ()

  Utilities for memory management and batch processing.
- integer function, public mod_memory::calculate_max_batch_size (available_memory, nb_total_encounters, reserve_percent)

  Calculate maximum batch size for matrix processing.

11.12 mod_pdb.f90 File Reference

Data Types

- type mod_pdb::type_pdb_file

  Data type for storing binned data (eg.
- type mod_pdb::type_pdb_chain
- type mod_pdb::type_pdb_residue
- type mod_pdb::type_pdb_atom

Modules

- module mod_pdb

  Module to define a pdb type and pdb reading routine for tools.

Functions/Subroutines

- subroutine mod_pdb::allocate_pdb_object (this, pdb_file, natoms, nresidues, nchains)

  Allocate and initialize a `type_pdb_file` structure from a PDB file.
- integer function mod_pdb::count_atoms (pdb_file)

  Count ATOM/HETATM records in a PDB file.
- integer function mod_pdb::count_residues (pdb_file)

  Count residues in a PDB file.
- integer function mod_pdb::count_chains (pdb_file)

  Count chain separators (TER records) in a PDB file.
- subroutine mod_pdb::fill_pdb_object (this, pdb_file)

  Parse PDB records and populate the `type_pdb_file` structure.

11.13 mod_threshold.f90 File Reference

Modules

- module mod_threshold

Functions/Subroutines

- subroutine mod_threshold::array_z_coord (array, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, solute_crds)

  Compute the Z-coordinate (along transformed z-axis) for each encounter.
- subroutine mod_threshold::array_atoms_dist (array, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, solute1↩ _crds, solute2_crds)

  Compute minimum inter-atomic distance between two solutes for each encounter.
- subroutine mod_threshold::array_angle (array, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, point1a, point1b, point2a, point2b, dimensions)

  Compute angle between two vectors (defined by atom groups) per encounter.
- subroutine mod_threshold::sort_array (arr, encounter_indexes)

  Sort an array in ascending order and reorder encounter indexes accordingly.

11.14 read_input.f90 File Reference

Modules

- module read_input

Functions/Subroutines

- subroutine read_input::read_pdb (pdb, filename, tot_atoms, tot_residues, tot_chains)

  Read a PDB file into a `type_pdb_file` object.
- subroutine read_input::read_assoc (assoc, filename, tot_encounters)

  Read an association/complexes file into a `type_assoc_file` object.

11.15 threshold.f90 File Reference

Functions/Subroutines

- program main

  Generate encounter selection based on threshold criteria.
- subroutine print_help (help_option)

  Print usage/help text for `threshold`*.*
- subroutine read_atoms_coord (atoms_array, atoms_coord, tot_atoms, pdb, pdb_filename)

  Read coordinates for a specified list of atoms (names or indices).
- subroutine write_cutoff_complexes (complexes_arg, distarray_arg, encounter_indexes_arg, nb_↩ encounters_arg, cutoff_arg, output_name_arg)

  Write selected encounter complexes to an output file.
- subroutine write_array (array_arg, filename_arg, cutoff_arg)

  Write array values up to a cutoff to a formatted file.

11.15.1 Function/Subroutine Documentation

11.15.1.1 main()

```
program main
```

Generate encounter selection based on threshold criteria.
This executable computes thresholds (z_coord, atoms_dist, angles) across encounter transforms and writes selected encounter records to an output complexes file. See `print_help` for usage examples.

Author

Abraham Muñiz-Chicharro

11.15.1.2 print_help()

```
subroutine main::print_help (
            character(len=*), intent(in) help_option )
```

Print usage/help text for `threshold`.
Explains command-line options and provides examples per threshold type.

11.15.1.3 read_atoms_coord()

```
subroutine main::read_atoms_coord (
            character(len=*), dimension(:), intent(in) atoms_array,
            real(kind=8), dimension(:, :), intent(out), allocatable atoms_coord,
            integer, intent(in) tot_atoms,
            type ( type_pdb_file ), intent(in) pdb,
            character(len=*), intent(in) pdb_filename )
```

Read coordinates for a specified list of atoms (names or indices).
Similar to the helper in `make_matrix`. Returns an allocated `atoms_coord` array with matching coordinates for the atom names or indices provided in `atoms_array`.

11.15.1.4 write_array()

```
subroutine main::write_array (
            real(kind=8), dimension(:), intent(in) array_arg,
            character*128, intent(in) filename_arg,
            real(kind=8), intent(in) cutoff_arg )
```

Write array values up to a cutoff to a formatted file.
Writes values from `array` (in ascending order) until `cutoff` is reached and stores them in `filename` using `F10.4` formatting.

11.15.1.5 write_cutoff_complexes()

```
subroutine main::write_cutoff_complexes (
            type(type_assoc_file), intent(in) complexes_arg,
            real (kind=8), dimension(:), intent(in) distarray_arg,
            integer, dimension(:), intent(in) encounter_indexes_arg,
            integer, intent(in) nb_encounters_arg,
            real(kind=8), intent(in) cutoff_arg,
            character*128, intent(in) output_name_arg )
```

Write selected encounter complexes to an output file.
Writes the header lines from `complexeshead` and then appends encounter lines (from `complexeslines`) in the order provided by `encounter_indexes` until a value exceeding `cutoff` is encountered.

11.16 unit_test/INDEX.md File Reference

11.17 unit_test/QUICK_TEST.md File Reference

11.18 unit_test/README.md File Reference

11.19 unit_test/run_all_tests.sh File Reference

11.20 unit_test/test_assoc.f90 File Reference

Functions/Subroutines

- program test_assoc

  Unit tests for reading association/complexes files.
- subroutine test_read_assoc_file (num_tests, num_passed, num_failed)
- subroutine test_allocate_assoc_object (num_tests, num_passed, num_failed)
- subroutine test_size_assoc (num_tests, num_passed, num_failed)

11.20.1 Function/Subroutine Documentation

11.20.1.1 test_allocate_assoc_object()

```
subroutine test_assoc::test_allocate_assoc_object (
            integer, intent(inout) num_tests,
            integer, intent(inout) num_passed,
            integer, intent(inout) num_failed )
```

11.20.1.2 test_assoc()

```
program test_assoc
```

Unit tests for reading association/complexes files.
Comprehensive test suite for reading and parsing association/complexes files in the read_input and mod_assoc modules including:

- **File reading**:
  - Valid assoc file reading
  - File existence checking
  - Error handling for missing files
  - Correct number of encounters parsed

- **Data structure allocation**:
  - Allocate assoc objects with correct sizes
  - Initialize translation vectors to zero
  - Initialize rotation vectors (rot1, rot2) to zero
  - Allocate lines array
- **File parsing**:
  - Skip comment lines (starting with #)
  - Count data lines correctly

Author

Abraham Muñiz-Chicharro

Version

1.0

- Extract center coordinates (xc1, xc2)
- Parse translation vectors
- Parse rotation vectors

- **Data integrity**:
  - Verify allocations match encounter count
  - Check header information is stored
  - Verify vector values are reasonable
  - Ensure no data corruption during read

Compile with: gfortran -o test_assoc test_assoc.f90 ../src/read_input.f90 ../src/mod_assoc.f90

Author

Abraham Muñiz-Chicharro

Version

1.0

11.20.1.3 test_read_assoc_file()

```
subroutine test_assoc::test_read_assoc_file (
             integer, intent(inout) num_tests,
             integer, intent(inout) num_passed,
             integer, intent(inout) num_failed )
```

11.20.1.4 test_size_assoc()

```
subroutine test_assoc::test_size_assoc (
             integer, intent(inout) num_tests,
             integer, intent(inout) num_passed,
             integer, intent(inout) num_failed )
```

11.21 unit_test/test_clustering.f90 File Reference

Functions/Subroutines

- program test_clustering

  Unit tests for clustering algorithms in the mod_clust_algorithm module.
- subroutine test_clustering_statistics (num_tests, num_passed, num_failed)
- subroutine test_simple_clustering (num_tests, num_passed, num_failed)
- subroutine test_minimum_linkage (num_tests, num_passed, num_failed)
- subroutine test_maximum_linkage (num_tests, num_passed, num_failed)
- subroutine test_mean_linkage (num_tests, num_passed, num_failed)
- subroutine test_cluster_writing (num_tests, num_passed, num_failed)

11.21.1 Function/Subroutine Documentation

11.21.1.1 test_cluster_writing()

```
subroutine test_clustering::test_cluster_writing (
            integer, intent(inout) num_tests,
            integer, intent(inout) num_passed,
            integer, intent(inout) num_failed )
```

11.21.1.2 test_clustering()

```
program test_clustering
```

Unit tests for clustering algorithms in the mod_clust_algorithm module.
Tests include:

- Sort complexes functionality
- Write cluster elements
- Basic clustering threshold calculation
- Minimum linkage clustering
- Maximum linkage clustering
- Mean linkage clustering

Note: Array parameter is mandatory for all clustering methods EXCEPT RMSD. For RMSD-based clustering, array is optional. When provided, array is used to compute cluster averages and standard deviations.
Compile with: gfortran -fopenmp -o test_clustering test_clustering.f90 ../src/mod_clust_algorithm.f90 ../src/read_↩ input.f90

Author

Abraham Muniz-Chicharro

Version

1.0

11.21.1.3 test_clustering_statistics()

```
subroutine test_clustering::test_clustering_statistics (
            integer, intent(inout) num_tests,
            integer, intent(inout) num_passed,
            integer, intent(inout) num_failed )
```

11.21.1.4 test_maximum_linkage()

```
subroutine test_clustering::test_maximum_linkage (
             integer, intent(inout) num_tests,
             integer, intent(inout) num_passed,
             integer, intent(inout) num_failed )
```

11.21.1.5 test_mean_linkage()

```
subroutine test_clustering::test_mean_linkage (
             integer, intent(inout) num_tests,
             integer, intent(inout) num_passed,
             integer, intent(inout) num_failed )
```

11.21.1.6 test_minimum_linkage()

```
subroutine test_clustering::test_minimum_linkage (
             integer, intent(inout) num_tests,
             integer, intent(inout) num_passed,
             integer, intent(inout) num_failed )
```

11.21.1.7 test_simple_clustering()

```
subroutine test_clustering::test_simple_clustering (
             integer, intent(inout) num_tests,
             integer, intent(inout) num_passed,
             integer, intent(inout) num_failed )
```

11.22 unit_test/test_maths.f90 File Reference

Functions/Subroutines

- program test_maths

  Unit tests for mathematical operations in the maths module.
- subroutine test_cross_product (num_tests, num_passed, num_failed)
- subroutine test_update_complex (num_tests, num_passed, num_failed)
- subroutine test_vectors_angle_3d (num_tests, num_passed, num_failed)
- subroutine test_vectors_angle_2d (num_tests, num_passed, num_failed)
- subroutine test_rmsd (num_tests, num_passed, num_failed)
- subroutine test_calculate_cog (num_tests, num_passed, num_failed)
- subroutine test_calculate_distance (num_tests, num_passed, num_failed)
- logical function assert_vector_equal (v1, v2, tolerance)

  Helper function: Check if two vectors are approximately equal.

11.22.1 Function/Subroutine Documentation

11.22.1.1 assert_vector_equal()

```
logical function test_maths::assert_vector_equal (
             real(kind=8), dimension(3), intent(in) v1,
             real(kind=8), dimension(3), intent(in) v2,
             real(kind=8), intent(in) tolerance )
```

Helper function: Check if two vectors are approximately equal.

11.22.1.2 test_calculate_cog()

```
subroutine test_maths::test_calculate_cog (
             integer, intent(inout) num_tests,
             integer, intent(inout) num_passed,
             integer, intent(inout) num_failed )
```

11.22.1.3 test_calculate_distance()

```
subroutine test_maths::test_calculate_distance (
             integer, intent(inout) num_tests,
             integer, intent(inout) num_passed,
             integer, intent(inout) num_failed )
```

11.22.1.4 test_cross_product()

```
subroutine test_maths::test_cross_product (
             integer, intent(inout) num_tests,
             integer, intent(inout) num_passed,
             integer, intent(inout) num_failed )
```

11.22.1.5 test_maths()

```
program test_maths
```

Unit tests for mathematical operations in the maths module.
Comprehensive test suite for all subroutines in the maths module including:

- **Cross product**:
  - Basis vectors (i x j = k)
  - Reverse order (anticommutative property)
  - Self cross product (v x v = 0)
  - Arbitrary vectors
- **Coordinate transformations (update_complex)**:
  - Identity transformations
  - Pure translation
  - Only xc1 translation
  - Only xc2 translation
  - Only rotation (90 degrees)
  - Combined transformations
- **3D angle calculations (vectors_angle_3D)**:
  - Parallel vectors (0 degrees)

Author

Abraham Muñiz-Chicharro

Version

1.0

- Perpendicular vectors (90 degrees)
- Opposite vectors (180 degrees)
- 45 degree angles

- **2D angle calculations (vectors_angle_2D)**:
  - Parallel vectors in XY plane (0 degrees)
  - Perpendicular vectors in XY plane (90 degrees)
  - Opposite vectors in XY plane (180 degrees)
  - Zero magnitude vectors handling
- **RMSD calculation**:
  - Identical coordinates (RMSD = 0)
  - Simple translations
  - Known RMSD values
- **Center of geometry (calculate_cog)**:
  - Single point
  - Points at origin
  - Symmetric point distributions
  - General cases
- **Distance calculation (calculate_distance)**:
  - Distance to self (0)
  - Unit distances
  - 3D diagonal distances
  - Arbitrary point pairs

Compile with: gfortran -o test_maths test_maths.f90 ../src/maths.f90 ../src/mod_pdb.f90

Author

Abraham Muñiz-Chicharro

Version

1.0

11.22.1.6 test_rmsd()

```
subroutine test_maths::test_rmsd (
             integer, intent(inout) num_tests,
             integer, intent(inout) num_passed,
             integer, intent(inout) num_failed )
```

11.22.1.7 test_update_complex()

```
subroutine test_maths::test_update_complex (
             integer, intent(inout) num_tests,
             integer, intent(inout) num_passed,
             integer, intent(inout) num_failed )
```

11.22.1.8 test_vectors_angle_2d()

```
subroutine test_maths::test_vectors_angle_2d (
             integer, intent(inout) num_tests,
             integer, intent(inout) num_passed,
             integer, intent(inout) num_failed )
```

11.22.1.9 test_vectors_angle_3d()

```
subroutine test_maths::test_vectors_angle_3d (
             integer, intent(inout) num_tests,
             integer, intent(inout) num_passed,
             integer, intent(inout) num_failed )
```

11.23 unit_test/test_matrix.f90 File Reference

Functions/Subroutines

- program test_matrix

  Unit tests for matrix operations in the mod_matrix module.
- subroutine test_write_read_matrix (num_tests, num_passed, num_failed)
- subroutine test_write_read_array (num_tests, num_passed, num_failed)
- subroutine test_matrix_z_coord (num_tests, num_passed, num_failed)
- subroutine test_matrix_rmsd_calc (num_tests, num_passed, num_failed)
- subroutine test_matrix_atoms_dist (num_tests, num_passed, num_failed)
- subroutine test_matrix_angle_calc (num_tests, num_passed, num_failed)

11.23.1 Function/Subroutine Documentation

11.23.1.1 test_matrix()

```
program test_matrix
```

Unit tests for matrix operations in the mod_matrix module.
Tests include:

- Write and read matrix operations
- Write and read array operations
- Matrix Z-coordinate calculation
- Matrix atoms distance calculation
- Matrix angle calculation
- RMSD matrix calculation

Compile with: gfortran -fopenmp -o test_matrix test_matrix.f90 ../src/mod_matrix.f90 ../src/maths.f90 ../src/mod_↩ pdb.f90

Author

Abraham Muñiz-Chicharro

Version

1.0

11.23.1.2 test_matrix_angle_calc()

```
subroutine test_matrix::test_matrix_angle_calc (
             integer, intent(inout) num_tests,
             integer, intent(inout) num_passed,
             integer, intent(inout) num_failed )
```

11.23.1.3 test_matrix_atoms_dist()

```
subroutine test_matrix::test_matrix_atoms_dist (
             integer, intent(inout) num_tests,
             integer, intent(inout) num_passed,
             integer, intent(inout) num_failed )
```

11.23.1.4 test_matrix_rmsd_calc()

```
subroutine test_matrix::test_matrix_rmsd_calc (
             integer, intent(inout) num_tests,
             integer, intent(inout) num_passed,
             integer, intent(inout) num_failed )
```

11.23.1.5 test_matrix_z_coord()

```
subroutine test_matrix::test_matrix_z_coord (
             integer, intent(inout) num_tests,
             integer, intent(inout) num_passed,
             integer, intent(inout) num_failed )
```

11.23.1.6 test_write_read_array()

```
subroutine test_matrix::test_write_read_array (
             integer, intent(inout) num_tests,
             integer, intent(inout) num_passed,
             integer, intent(inout) num_failed )
```

11.23.1.7 test_write_read_matrix()

```
subroutine test_matrix::test_write_read_matrix (
             integer, intent(inout) num_tests,
             integer, intent(inout) num_passed,
             integer, intent(inout) num_failed )
```

11.24 unit_test/test_pdb.f90 File Reference

Functions/Subroutines

- program test_pdb

  Unit tests for PDB operations in the mod_pdb module.
- subroutine test_pdb_allocation (num_tests, num_passed, num_failed)
- subroutine test_atom_properties (num_tests, num_passed, num_failed)
- subroutine test_residue_management (num_tests, num_passed, num_failed)
- subroutine test_chain_management (num_tests, num_passed, num_failed)
- subroutine test_mod_pdb_routines (num_tests, num_passed, num_failed)

11.24.1 Function/Subroutine Documentation

11.24.1.1 test_atom_properties()

```
subroutine test_pdb::test_atom_properties (
             integer, intent(inout) num_tests,
             integer, intent(inout) num_passed,
             integer, intent(inout) num_failed )
```

11.24.1.2 test_chain_management()

```
subroutine test_pdb::test_chain_management (
             integer, intent(inout) num_tests,
             integer, intent(inout) num_passed,
             integer, intent(inout) num_failed )
```

11.24.1.3 test_mod_pdb_routines()

```
subroutine test_pdb::test_mod_pdb_routines (
             integer, intent(inout) num_tests,
             integer, intent(inout) num_passed,
             integer, intent(inout) num_failed )
```

11.24.1.4 test_pdb()

```
program test_pdb
```

Unit tests for PDB operations in the mod_pdb module.
Tests include:

- PDB type allocation and initialization
- Atom and residue management
- Residue structure handling
- Reading PDB files and filling the PDB data structure

Compile with: gfortran -o test_pdb test_pdb.f90 mod_pdb.f90

Author

Abraham Muñiz-Chicharro

Version

1.0

11.24.1.5 test_pdb_allocation()

```
subroutine test_pdb::test_pdb_allocation (
             integer, intent(inout) num_tests,
             integer, intent(inout) num_passed,
             integer, intent(inout) num_failed )
```

11.24.1.6 test_residue_management()

```
subroutine test_pdb::test_residue_management (
             integer, intent(inout) num_tests,
             integer, intent(inout) num_passed,
             integer, intent(inout) num_failed )
```

11.25 unit_test/test_read_input.f90 File Reference

Functions/Subroutines

- program test_read_input

  Unit tests for the read_input module.
- subroutine test_read_input_assoc (num_tests, num_passed, num_failed)
- subroutine test_error_handling (num_tests, num_passed, num_failed)

11.25.1 Function/Subroutine Documentation

11.25.1.1 test_error_handling()

```
subroutine test_read_input::test_error_handling (
             integer, intent(inout) num_tests,
             integer, intent(inout) num_passed,
             integer, intent(inout) num_failed )
```

11.25.1.2 test_read_input()

```
program test_read_input
```

Unit tests for the read_input module.
Comprehensive test suite for reading PDB and association/complexes files using the read_input module wrapper functions including:

- **PDB file reading (read_pdb)**:
  - Valid PDB file reading
  - File existence checking
  - Atom count verification
  - Residue count verification
  - Chain count verification
  - Proper structure allocation
  - Error handling for missing files
- **Association file reading (read_assoc)**:
  - Valid assoc file reading
  - Encounter count verification
  - Structure allocation and initialization
  - Vector data integrity
  - Error handling for missing files

Author

Abraham Muñiz-Chicharro

Version

1.0

- **Integration tests**:
  - Sequential reading of multiple files
  - Proper cleanup between reads
  - Memory management

Compile with: gfortran -o test_read_input test_read_input.f90 ../src/read_input.f90 ../src/mod_pdb.f90 ../src/mod_assoc.f90

Author

Abraham Muñiz-Chicharro

Version

1.0

11.25.1.3 test_read_input_assoc()

```
subroutine test_read_input::test_read_input_assoc (
             integer, intent(inout) num_tests,
             integer, intent(inout) num_passed,
             integer, intent(inout) num_failed )
```

11.26 unit_test/TEST_SUMMARY.md File Reference

11.27 unit_test/test_threshold.f90 File Reference

Functions/Subroutines

- program test_threshold

  Unit tests for threshold operations in the mod_threshold module.
- subroutine test_array_z_coord (num_tests, num_passed, num_failed)
- subroutine test_array_atoms_dist (num_tests, num_passed, num_failed)
- subroutine test_array_angle (num_tests, num_passed, num_failed)
- subroutine test_sort_array (num_tests, num_passed, num_failed)

11.27.1 Function/Subroutine Documentation

11.27.1.1 test_array_angle()

```
subroutine test_threshold::test_array_angle (
             integer, intent(inout) num_tests,
             integer, intent(inout) num_passed,
             integer, intent(inout) num_failed )
```

11.27.1.2 test_array_atoms_dist()

```
subroutine test_threshold::test_array_atoms_dist (
             integer, intent(inout) num_tests,
             integer, intent(inout) num_passed,
             integer, intent(inout) num_failed )
```

11.27.1.3 test_array_z_coord()

```
subroutine test_threshold::test_array_z_coord (
             integer, intent(inout) num_tests,
             integer, intent(inout) num_passed,
             integer, intent(inout) num_failed )
```

11.27.1.4 test_sort_array()

```
subroutine test_threshold::test_sort_array (
             integer, intent(inout) num_tests,
             integer, intent(inout) num_passed,
             integer, intent(inout) num_failed )
```

11.27.1.5 test_threshold()

```
program test_threshold
```

Unit tests for threshold operations in the mod_threshold module.
Tests include:

- Z-coordinate array calculation
- Minimum distance calculation between atoms
- Angle computation for different dimensions
- Array sorting with encounter index tracking

Compile with: gfortran -fopenmp -o test_threshold test_threshold.f90 ../src/mod_threshold.f90 ../src/maths.f90 ../src/mod_pdb.f90

Author

Abraham Muñiz-Chicharro

Version

1.0

11.28 unit_test/TESTING.md File Reference

Index